Master Team Project Summer 2024
ClassCompanion
Milestone 5

Team 1

Date: 13.09.2024

Content and structure

1. Milestone 1
2. Milestone 2
3. Milestone 4
4. ClassCompanion Screenshots

Master Team Project Summer 2024
ClassCompanion
Milestone 1

Team 1

Yagubkhan Yagubkhanov, Team Lead / Github Master
yagubkhan.yagubkhanov@informatik.hs-fulda.de

Samman Adhikari, Frontend Lead

Shantanu Patil, Backend Lead

Aralikere Yogananda Gunarachana

Mariha Asif

Amber Asad

Date: 19.05.2024
Date Submitted: 17.05.2024
Date Revised: 19.05.2024

Revision Summary:

Date	Revision	Summary
13.05.2024	1.0	Initial draft created
17.05.2024	1.1	Revised use cases and competitive analysis, added unique features
19.05.2024	1.2	USP added, added 1 more persona, revised list of main data items, fixed some minor issues

1.Executive Summary:

The motivation behind the development of our application is found in the crucial need for accessible and effective academic support within the Fulda University of Applied Sciences community. With a diverse range of users, including students, tutors, parents, administrators, and moderators, a seamless and secure environment is aimed at connecting students with tutors, facilitating learning, and fostering collaboration. By offering tailored services, ensuring user privacy, and adhering to established security protocols, the academic experience is aimed at being enhanced, and academic success for students is aimed at being promoted while also streamlining administrative tasks for educators and administrators.

A comprehensive suite of functions and services tailored to the needs of our diverse user base will be provided by our application. Students will be able to create profiles, search for tutors based on various criteria, engage in live chat sessions, and provide feedback through ratings and reviews. Tutors, on the other hand, will have the ability to create detailed profiles showcasing their expertise, offer tutoring services, and communicate with students. Tools for managing user accounts, content moderation, and ensuring platform integrity will be available for administrators. By prioritizing ease of use, security, and scalability, a vibrant learning community is aimed at being fostered that promotes knowledge sharing and academic growth while providing essential support for all stakeholders within the university ecosystem. We offer unique features like Group Chat for collaborative learning, Calendar Integration for scheduling, and a powerful admin panel for efficient management. It's tailored to enhance learning and administrative processes.

Our student startup team comprises six members, each bringing a unique blend of experience and fresh perspectives to the table. While most of us have significant experience in the industry, a few are new to it, creating a balanced mix of seasoned professionals and fresh talent. This diversity allows us to approach challenges from different angles, drawing on both the knowledge gained from years of industry experience and the innovative ideas brought by newcomers. Together, we form a dynamic team capable of tackling complex problems and driving our startup forward with creativity and expertise.

2. **Personae and main Use Cases**:

Personas

Persona 1 (Student)

John Smith is a student at Hoschule Fulda and he is pursuing Bachelors of Science in Applied Computer Science. Besides his regular lecture and self study, he often finds himself struggling with concepts and needing someone else's help. So, to make sure he no longer wastes his time, he has decided to seek some professional help. He is now looking for some tutor online who can provide him direct one to one guidance.

Goals:

- Discover Computer Science Tutors: John wants to find some tutors online who can help him with his computer science studies.
- Convenient Communication: Johns wants a very convenient platform to connect to those tutors and share resources and ideas.

Pain Points:

- Low Budget: Being a student, John is tight on budget.
- Concerns About Finding A Suitable Tutor: Since every tutor has his/her own style, finding a tutor who can match John's style of learning can be difficult.

Persona 2 (Tutor)

Sara Brown is a professional Software Engineer, with around 2 years of experience. Besides building software, Sara also loves to teach. So, she has decided to become an online tutor for computer science students, where she will be able to earn some extra cash doing what she loves.

Goals:

- Market Herself As A Tutor: Sara wants to showcase her skills and market herself, so that she can reach students who she will be able to help.
- Convenient Communication: Sara also wants a convenient platform to connect to her students.

Pain Points:

- Limited Time: Since Sara has a full-time job, she has very limited time.

Persona 3 (Guest)

Tim Koch is a student at Hochschule Fulda, studying Digital Media (BSc.). He is struggling with some of his courses, and hence needs some help. He has decided to look for some online tutors. He has recently discovered a new tutoring app "Class Companion" from his university's notice board. Since he saw that this app was specifically designed for Hochschule Fulda students, he decided to give it a shot. But, he is a bit cautious when it comes to his privacy. He does not want to immediately register for the application, without exploring it a bit and seeing if he can actually find some suitable tutors.

Goals:

- Find some suitable tutors: Tim wants to search some tutors for his classes and see if there are any who piques his interest.
- Use the application as a guest: Since Tim is a bit cautious when it comes to sharing his personal information like email, he first wants to see if there are any well reviewed, suitable tutors, before registering.

Pain Points:

- Cautious when it comes to sharing personal information: Tim does not like to register for any websites/apps without being totally sure that he will need the services.

Use Cases

1. **Discover suitable tutor - for John Smith (student)**
   John will login to the application and directly type in the course name that he wants help with. He will also apply filters, such as years of experience, languages etc. Then, he will see and evaluate the listed tutors based on quick information like the rating, price (hourly/per question) etc. If John sees any tutor that piques his interest, he will click on that tutor's link to open the details page. There he will see details such as the tutor's bio, reviews etc. Then John will be able to send a request to connect, including an initial message to the tutor.

2. **Create your profile - for Sara Brown (tutor)**
   Sara will come to our application to register as a tutor. She will first enter basic information like their name, email address etc. Then she will enter some more information like bio, their field of expertise, years of experience etc. This additional information will be crucial to market Sara to suitable students. Then finally Sara will be able to enter her desired hourly class rate and maybe even query rates (for example 10 query answers through text chat for x euros). Now, she will just wait for students to send her connection requests.

3. **Ask tutor's help - for John Smith (student)**
   John has a problem that he needs help with. He will go to his chat page. There John will message the tutor directly asking help for the question he has. John will then wait for the tutor to answer his question.

4. **Guest access - for Martin Wilson (guest)**
   Martin, a new interested student, first opens our application. He will have the option to wander in a few pages of the app without registering. Martin will first be greeted with some of the highest rated tutors of different fields. He will also be able to search for a tutor of any specific area (e.g. Maths). Martin will then see a list of highest rated math tutors. In the same list he will also see important information like their rating, quick bio etc. Then when Martin finds a suitable tutor and clicks on it, he will be redirected to the registration page.

3.**List of main data items and entities**:

1. Type of Users:

Student: A registered user looking for tutoring assistance.
Tutor: People who provide tutoring services.
Administrator: In charge of overseeing and approving the profiles of tutors and keeping the website up to date.
Guest: Non-registered visitors can view tutor profiles and obtain basic service information.
Parent/Guardian: Individuals who oversee student accounts; particularly pertinent for students who are not yet of legal age.

2. Student/Tutor Profile:

Comprehensive information about all the registered students and tutors using the website including personal information, contact details, qualifications, experiences, academic details, and preferences.

3. Messages:

Students will be able to communicate with tutors and each other.

4. Available Services:

Complete information about all the courses offered, and their fees, duration, availability of tutors etc.

5. Feedback:

Student ratings of tutors and reviews about their tutoring experiences.

6. Administration:

Features that are provided for website administrator supervise tutor profiles, reviews, user accounts, and platform content quality.

7. Search Bar:

By simply typing keywords, students will be able to look for and select teachers by subject, availability, rating, etc.

8. Multimedia:

Tutors can upload their CV, files, lectures after administrator approval.

4.**Initial list of functional requirements**:

1. **User Registration and Login:**
    - Students and tutors must register using their university email address.
    - Check box to register as student or tutor
    - Registration process should include validation of email addresses to ensure eligibility.
    - Registered users (students and tutors) should be able to log in and log out securely.
2. **Profile Management:**
    - Users (students and tutors) should be able to manage their profiles, including updating personal information, adding skills, and uploading profile pictures.
    - Students should not be able to search for other students based on skills or subjects in tutor search.
3. **Tutor Search Functionality:**
    - Students should be able to search for tutors based on skills, subjects, or keywords.
    - Search results should display a list of relevant tutors based on the search criteria.
4. **Live Chat Feature:**
    - Students should be able to initiate live chat sessions with tutors they want to contact.
    - Chat functionality should support text messages, file uploads (such as documents or images), and other multimedia content.
    - If a tutor is offline, messages should be delivered to their inbox for later viewing.
    - Group chat so a teacher can connect with multiple students interested in same topic simultaneously.
5. **Inbox Management:**
    - Both tutors and students should have access to their respective inboxes.
    - Inboxes should display messages received from other users, including chat messages and notifications.
    - Users should be able to mark messages as read, reply to messages, and delete messages as needed.
6. **Guest Access:**
    - Guest students and teachers should be allowed to access the website, but they cannot utilize chat functionality until they register with their university email address.
7. **Admin Panel:**
    - An admin panel should be available for administrators to manage user accounts, monitor chat activity, and handle any reported issues.
    - Admins should have the ability to view user profiles, moderate content, and enforce site policies.

5.List of non-functional requirements:

Mandatory Requirements:

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in Milestone 0. Application delivery shall be from chosen cloud server.
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers.
3. All or selected application functions must render well on mobile devices.
4. Data shall be stored in the database on the team's deployment cloud server.
5. No more than 50 concurrent users shall be accessing the application at any time. 6. Privacy of users shall be protected, and all privacy policies will be appropriately communicated to the users.
7. The language used shall be English (no localization needed).
8. Application shall be very easy to use and intuitive.
9. Application should follow established architecture patterns.
10. Application code and its repository shall be easy to inspect and maintain.
11. Google analytics shall be used (optional for Fulda teams).
12. No e-mail clients shall be allowed.
13. Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI.
14. Site security: basic best practices shall be applied (as covered in the class) for main data items.
15. Application shall be media rich (images, video etc.). Media formats shall be standard as used in the market today.
16. Modern SE processes and practices shall be used as specified in the class, including collaborative and continuous SW development.
17. For code development and management, as well as documentation like formal milestones required in the class, each team shall use their own GitHub to be set-up by class instructors and started by each team during Milestone 0.
18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "Fulda University of Applied Sciences Software Engineering Project, Fall 2021 For Demonstration Only" at the top of the WWW page. (Important so as to not confuse this with a real application).

1. Performance:

Response Time: To provide a seamless user experience, all platform activities, including finding tutors, loading tutor profiles, and sending questions, should take less than two seconds to complete.

Scalability: To handle growing user and data volumes without noticeably degrading performance, the system should be built to scale horizontally. This includes the capacity to increase the number of server instances by using cloud-based elastic scaling options.

Resource Utilization: To ensure smooth operation even under heavy loads, the application should be optimized to make the best use of the system's resources. This includes minimizing CPU and memory utilization.

2. Expected Load:

Peak Load Handling: To avoid downtime or performance difficulties, the platform must be able to handle peak loads during exam seasons, registration periods, and other periods of high demand.
Load Testing: To find bottlenecks and adjust system performance appropriately, load testing should be done regularly.

3. Security Requirements:

Data Encryption: To avoid unwanted access, all sensitive user data, such as payment card numbers and personal information, should be encrypted both in transit and at rest.
Authentication procedures: To guarantee that only authorized users may access the platform, strong authentication procedures like multi-factor authentication should be put in place.
Authorization: To limit access to sensitive features or data based on user roles, rolebased access control should be implemented.
Data privacy: It is important to guarantee adherence to data protection laws and to establish explicit guidelines for the handling, storing, and sharing of data.

4. Storage:

Data Retention: To guarantee legal compliance and maximize storage utilization, establish policies for data retention and archiving.
Backup and recovery: To guard against losing important data in the event of a cyberattack, hardware malfunction, or unintentional deletion, regular backups of that data should be made.
Scalable Storage: Without compromising performance or dependability, the storage system should be scalable to handle increasing data volumes over time.

5. Availability:

High Availability: To reduce downtime and guarantee availability of at least 99.9%, the platform should be built with redundancy and failover techniques.
Monitoring and Alerting: It is recommended to establish continuous monitoring of the system's uptime, performance metrics, and health. If any anomalies or errors are detected, an automatic alarm system should be set up.

6. Fault Tolerance:

Fault Detection: Implement mechanisms for detecting faults and errors in real-time, such as automated error logging and monitoring.

Fault Isolation: Isolate faults to prevent cascading failures that could affect other parts of the system, ensuring that issues are contained and localized.
Graceful Degradation: Design the system to gracefully degrade functionality in the event of failures, prioritizing core features and ensuring that users can still access essential services even during outages.

6.**Competitive analysis**:

Feature	**Competitor 1 (Tutor.com)**	**Competitor 2 (Chegg Tutors)**	**Competitor 3 (Wyzant)**	**Class Companion**
Registration Process	Email validation required	Email validation required	Email validation required	Email validation required
Live Chat	YES	YES	YES	YES
File Sharing	YES	YES	YES	YES
Profile Management	Limited	Limited	Limited	YES
Admin Panel	YES	YES	YES	YES
Group Chats	NO	NO	NO	YES
Calendar Integration	Limited	Limited	Limited	YES

Our planned product aims to offer a comprehensive online tutoring platform with features better than those of competitors such as Tutor.com, Chegg Tutors, and Wyzant. However, our platform distinguishes itself by requiring users to register using their university email address from HS Fulda, ensuring a more secure and tailored experience within the university community. Additionally, our platform prioritizes user-friendly profile management, allowing users to update personal information, add skills, and upload profile pictures easily.

We further enhance our platform with unique features such as Group Chat, enabling tutors and multiple students to discuss complex ideas and topics collaboratively. Our Calendar Integration feature allows tutors to show their availability, making it easy for students to schedule sessions. This includes creating meeting links and inviting all participants once a meeting is scheduled, ensuring everyone stays organized and connected.

The inclusion of a robust admin panel also sets our product apart, providing administrators with tools to manage user accounts, monitor chat activity, and enforce site policies effectively. Overall, our planned product aims to offer a seamless and efficient online tutoring experience, tailored specifically to the needs of the HS Fulda community.

7.High-level system architecture and technologies used:

Introduction

This section outlines the high-level system architecture and the technologies chosen for the development of our tutoring service website. The selection of these technologies is aimed at ensuring scalability, security, and performance while providing a seamless user experience.

Technologies Used

Backend Framework: Django
Description: Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It follows the Model-View-Controller (MVC) architectural pattern, providing robust tools for building web applications.
Justification: Django's built-in features such as authentication, ORM (Object-Relational Mapping), and admin panel make it well-suited to our project requirements.

Frontend Framework: React
Description: React is a JavaScript library for building user interfaces, developed by Facebook. It allows developers to create reusable UI components and efficiently manage the state of the application.
Justification: React's component-based architecture facilitates the creation of dynamic and interactive user interfaces, enhancing the overall user experience. Its virtual DOM (Document Object Model) enables efficient rendering, resulting in faster performance.

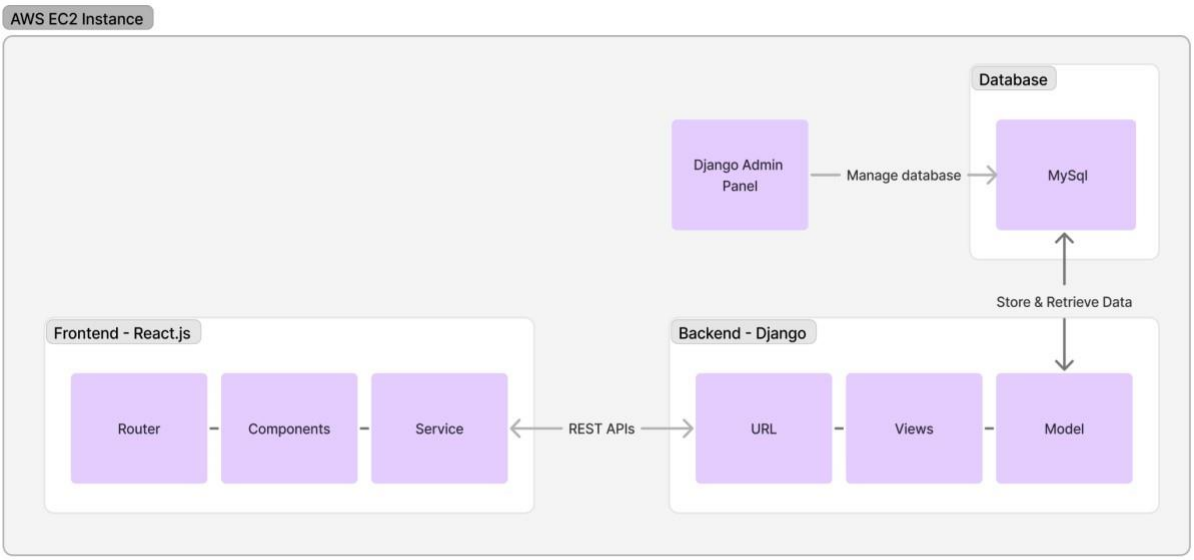
Database Management System: MySQL
Description: MySQL is an open-source relational database management system (RDBMS) known for its reliability, scalability, and performance. It provides robust features for managing structured data.
Justification: MySQL's widespread adoption, strong community support, and compatibility with Django make it an ideal choice for storing and managing various data entities in our application, including user profiles, messages, and feedback.

Cloud Platform: AWS (Amazon Web Services)
Description: AWS is a cloud computing platform offering a wide range of services, including computing power, storage, and databases, delivered as on-demand services over the internet.
Justification: AWS provides scalable and reliable infrastructure services, allowing us to deploy and manage our application with ease. Its comprehensive set of cloud services, including Amazon EC2 for hosting and Amazon RDS for MySQL database hosting, meets our project's requirements for scalability and reliability without any charges.

Additional Tools and Services

GitHub
Description: GitHub is a web-based platform for version control and collaboration, widely used by developers to host and review code, manage projects, and build software.
Justification: GitHub serves as our central repository for storing project code, enabling collaborative development among team members. Its version control features facilitate code management, branching, and merging, ensuring code quality and integrity.

Architecture Diagram



8.<u>Team and roles</u>:

Yagubkhan Yagubkhanov: Team Lead / Github Master / Document Master
Samman Adhikari: Frontend Lead
Shantanu Patil: Backend Lead
Aralikere Yogananda Gunarachana: Frontend
Mariha Asif: Frontend
Amber Asad: Backend

9. **Checklist**:

- Team found a time slot to meet (online) outside of the class: **DONE**
- GitHub master chosen : **DONE**
- Team decided and agreed together on using the listed SW tools and deployment server: **DONE**
- Team ready and able to use the chosen back and front end frameworks and those who need to learn are working on learning and practicing: **DONE**
- Team lead ensured that all team members read the final M1 and agree/understand it before submission: **DONE**
- GitHub organized as discussed in class (e.g. master branch, development branch, folder for milestone documents etc.): **DONE**

Master Team Project Summer 2024
ClassCompanion
Milestone 2

Team 1

Yagubkhan Yagubkhanov, Team Lead / Github Master
yagubkhan.yagubkhanov@informatik.hs-fulda.de

Samman Adhikari, Frontend Lead

Shantanu Patil, Backend Lead

Aralikere Yogananda Gunarachana

Mariha Asif

Amber Asad

Date: 10.06.2024

Revision Summary:

Date	Revision	Summary
27.05.2024	1.0	Initial draft created
04.06.2024	1.1	Made changes based on instructor's feedback, group chat feature added to USP as the priority 1
10.06.2024	1.2	Added new use cases and UI mockups

Content and structure for Milestone 2

1. Functional Requirements - Prioritized
2. List of main data items and entities
3. Use cases, UI Mockups and Storyboards
4. High level Architecture, Database Organization
5. High Level UML Diagrams
6. Key Risks for the Project Server Related Issues
7. Project management

1. Functional Requirements - Prioritized

Priority 1 Requirements (Must Have)

Registered Users (Students and Tutors)

1. **User Registration and Login**

   1.1 Students and tutors must register using their university email address.

   1.2 Registration process should include validation of email addresses to ensure eligibility.

   1.3 Registered users (students and tutors) should be able to log in and log out securely.

   1.4 Check box to register as student or tutor.

2. **Profile Management**

   2.1 Users (students and tutors) should be able to manage their profiles, including updating personal information, adding skills, and uploading profile pictures.

   2.2 Students should not be able to search for other students based on skills or subjects in tutor search.

3. **Tutor Search Functionality**

   3.1 Students should be able to search for tutors based on skills, subjects, or keywords.

   3.2 Search results should display a list of relevant tutors based on the search criteria.

4. **Live Chat Feature**

   4.1 Students should be able to initiate live chat sessions with tutors they want to contact.

   4.2 Chat functionality should support text messages, file uploads (such as documents or images), and other multimedia content.

   4.3 If a tutor is offline, messages should be delivered to their inbox for later viewing.

5. **Inbox Management**

   5.1 Both tutors and students should have access to their respective inboxes.

5.2 Inboxes should display messages received from other users, including chat messages and notifications.

5.3 Users should be able to mark messages as read, reply to messages, and delete messages as needed.

6. **Admin Panel**

   6.1 An admin panel should be available for administrators to manage user accounts, monitor chat activity, and handle any reported issues.

   6.2 Admins should have the ability to view user profiles, moderate content, and enforce site policies.

   6.3 Admins can moderate (hide or delete) inappropriate content. (USP)

7. **Group Chat Feature**

   7.1 Group chat functionality so a tutor can connect with multiple students interested in the same topic simultaneously.

Priority 2 Requirements (Desired)

Guest Users

1. **Guest Access**
   - Guest students and teachers should be allowed to access the website, but they cannot utilize chat functionality until they register with their university email address.

Priority 3 Requirements (Opportunistic)

Registered Users (Students and Tutors)

1. **Calendar Integration**
   - Integration of a calendar feature where tutors and students can schedule sessions and view availability.

Admin Panel

2. **Advanced Admin Features**
   - Additional tools for administrators to generate reports on usage statistics and user engagement.
   - Admins should be able to create announcements or broadcast messages to all users.

Summary

By organizing and prioritizing these requirements, we ensure that the most critical features necessary for the core functionality and user experience are developed first. This approach allows us to allocate resources effectively and iteratively improve the product by integrating additional features as time and resources permit.

2. List of main data items and entities

1. Type of Users:

Student

- ID: Unique identifier for each student.
- Name: Full name of the student.
- Username: Chosen username for login.
- Email: Email address provided by HS Fulda.
- Password: Encrypted password for account security.
- Contact Number: Phone number for communication.
- Address: Physical address.
- Date of Birth: For age verification.

Tutor

- ID: Unique identifier for each tutor.
- Name: Full name of the tutor.
- Username: Chosen username for login.
- Email: Email address provided by HS Fulda.
- Password: Encrypted password for account security.
- Contact Number: Phone number for communication.
- Address: Physical address.
- Academic Qualification: Highest level of education.
- Experience: Teaching experience and subject expertise.
- Rates: Hourly rates.
- Ratings: Average rating from student feedback.

Administrator

- ID: Unique identifier for each administrator.
- Name: Full name of the administrator.
- Username: Chosen username for login.
- Email: Email address provided by HS Fulda.
- Password: Encrypted password for account security.
- Contact Number: Phone number for communication.

- Roles: Specific administrative roles and access levels.

2. Student/Tutor Profile

- Personal Information: Name, gender, date of birth, and contact details.
- Contact Details: Phone number, email address, and physical address.
- Professional Experience: Highest level of Education, teaching experience and subject expertise (for tutors).
- Profile Picture: A photograph for profile identification.
- CV/Resume: Detailed professional document (for tutors).

3. Messages

- Message ID: Unique identifier for each message.
- Sender ID: User ID of the message sender.
- Receiver ID: User ID of the message receiver.
- Timestamp: Date and time when the message was sent.
- Content: Text content of the message.
- Status: Read or unread status.

4. Feedback

- Feedback ID: Unique identifier for each feedback entry.
- Student ID: User ID of the student providing feedback.
- Tutor ID: User ID of the tutor being reviewed.
- Rating: Numeric rating (e.g.1-5 stars).
- Timestamp: Date and time when the rating was submitted.

5. Administration

- User Management: Tools for managing students, tutor accounts.
- Content Moderation: Tools for moderating shared messages.

6. Search Functionality

- Search Query: Keywords entered by the user.
- Filters: Options to filter results by tutor rating and experience.
- Search Results: List of tutors or courses matching the search criteria.
- Sorting Options: Options to sort results by experience, rating, etc.

7. Multimedia

- Multimedia ID: Unique identifier for each multimedia item.
- Uploader ID: User ID of the tutor uploading the content.
- File Type: Type of file (e.g., PDF, video, audio).

- File Size: Size of the file.
- Upload Date: Date and time when the file was uploaded.

8. Groups

- Group ID: Unique identifier for each group entry.
- Name: Group name

9. Message Group

- Message Group ID: Unique identifier for each message group entry
- Group ID: Identifier of the associated group
- Message ID: Identifier of the associated message

3. Use Cases, UI Mockups and Storyboards

3.1 Use Cases

1. **Discover suitable tutor - for John Smith (student)**

John will login to the application and directly type in the course name that he wants help with. He will also apply filters, such as years of experience, languages etc. Then, he will see and evaluate the listed tutors based on quick information like the rating, price (hourly/per question) etc. If John sees any tutor that piques his interest, he will click on that tutor's link to open the details page. There he will see details such as the tutor's bio, reviews etc. Then John will be able to send a request to connect, including an initial message to the tutor.

2. **Create your profile - for Sara Brown (tutor)**

Sara will come to our application to register as a tutor. She will first enter basic information like their name, email address etc. Then she will enter some more information like bio, their field of expertise, years of experience etc. This additional information will be crucial to market Sara to suitable students. Then finally Sara will be able to enter her desired hourly class rate and maybe even query rates (for example 10 query answers through text chat for x euros). Now, she will just wait for students to send her connection requests.

3. **Ask tutor's help - for John Smith (student)**

John has a problem that he needs help with. He will go to his chat page. There John will message the tutor directly asking help for the question he has. John will then wait for the tutor to answer his question.

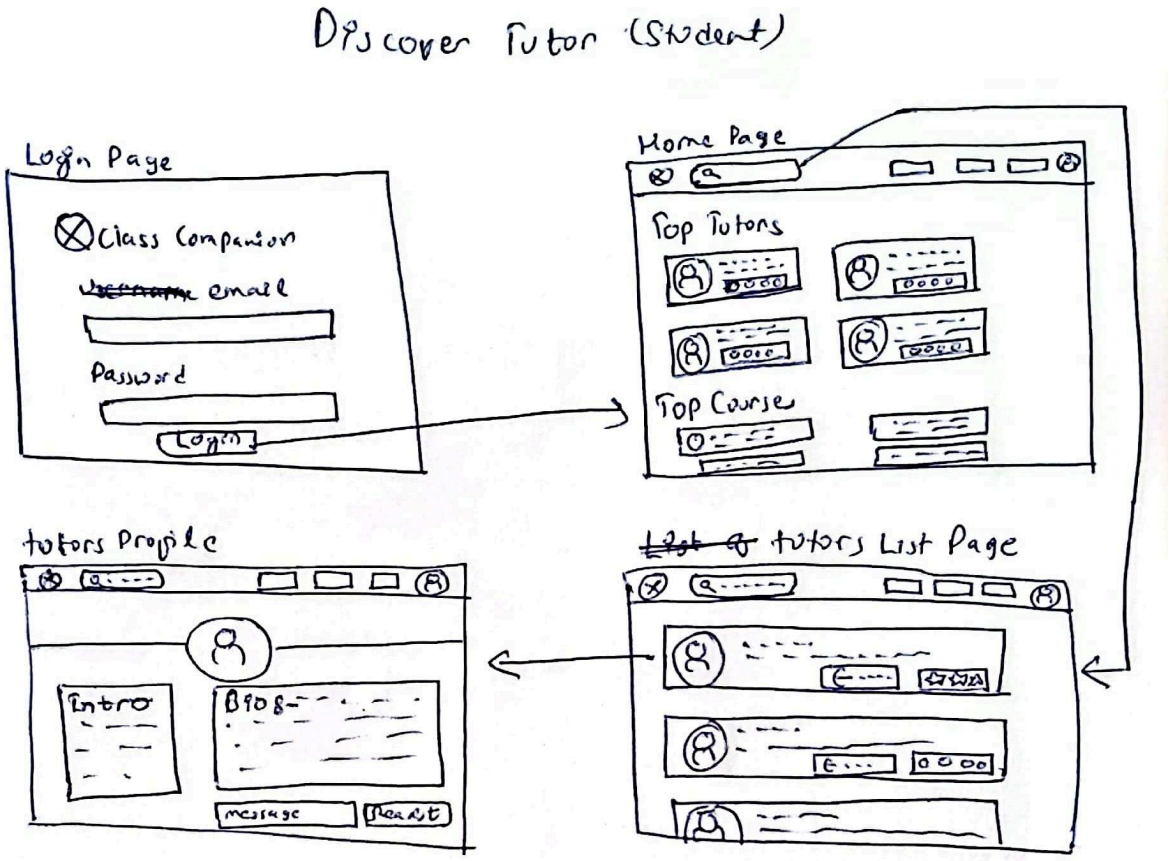
4. **Guest access - for Martin Wilson (guest)**

Martin, a new interested student, first opens our application. He will have the option to wander in a few pages of the app without registering. Martin will first be greeted with some of the highest rated tutors of different fields. He will also be able to search for a tutor of any specific area (e.g. Maths). Martin will then see a list of highest rated math tutors. In the same list he will also see important information like their rating, quick bio etc. Then when Martin finds a suitable tutor and clicks on it, he will be redirected to the registration page.

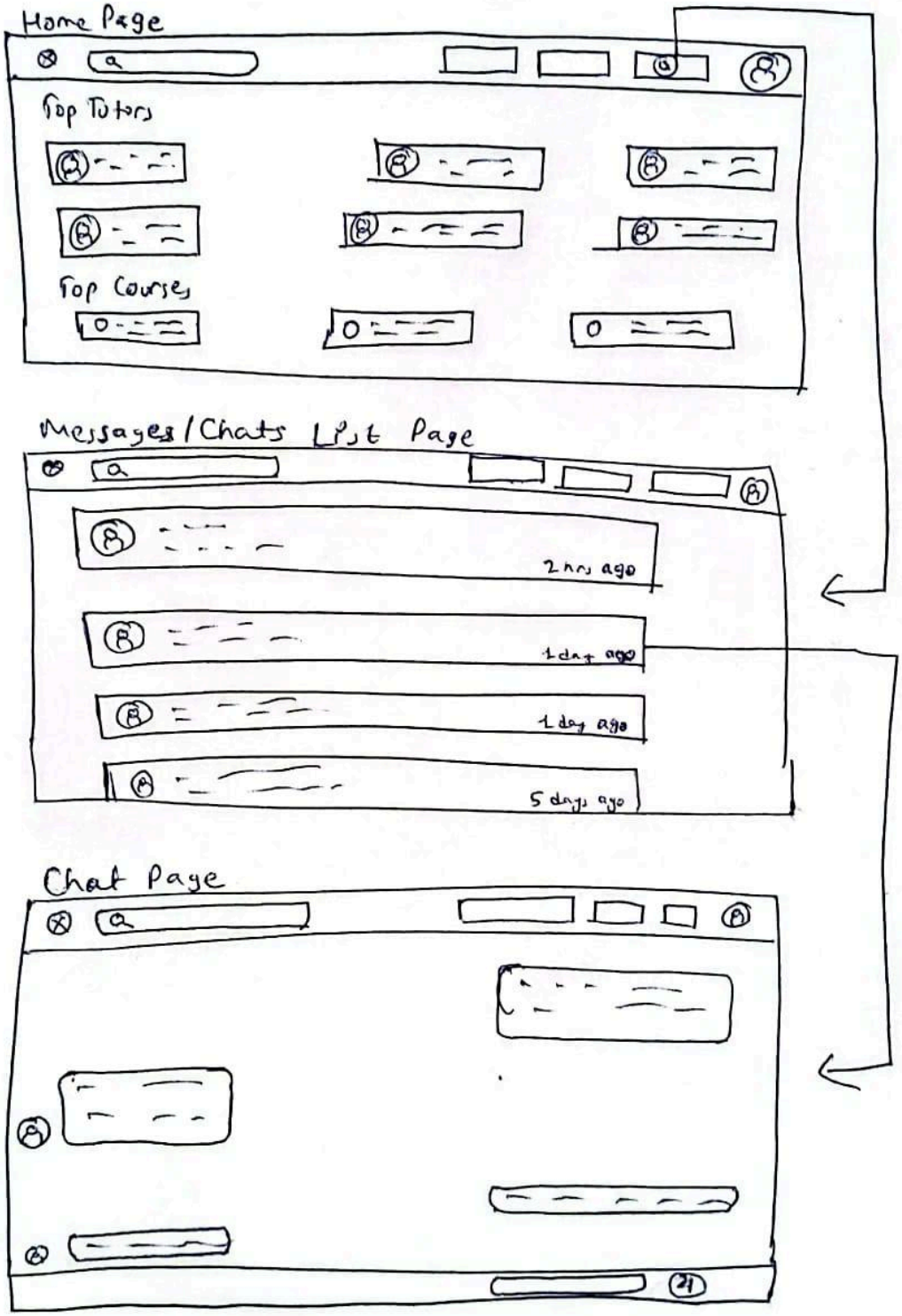
4. **Approve Tutor Application- for Administrator**

Administrator, wants to check if there are new reports/accounts in the admin dashboard that needs his attention. After logging in to the dashboard, he sees a list item that shows there is a new application for tutor that needs to be approved. He clicks on that item, which brings the admin to the applicant's detail page. Here he can check all the details of the person applying to become a tutor, including his/her CV. After making sure everything is in order, the admin will now hit the approve button to approve the application and create this new tutor's account.

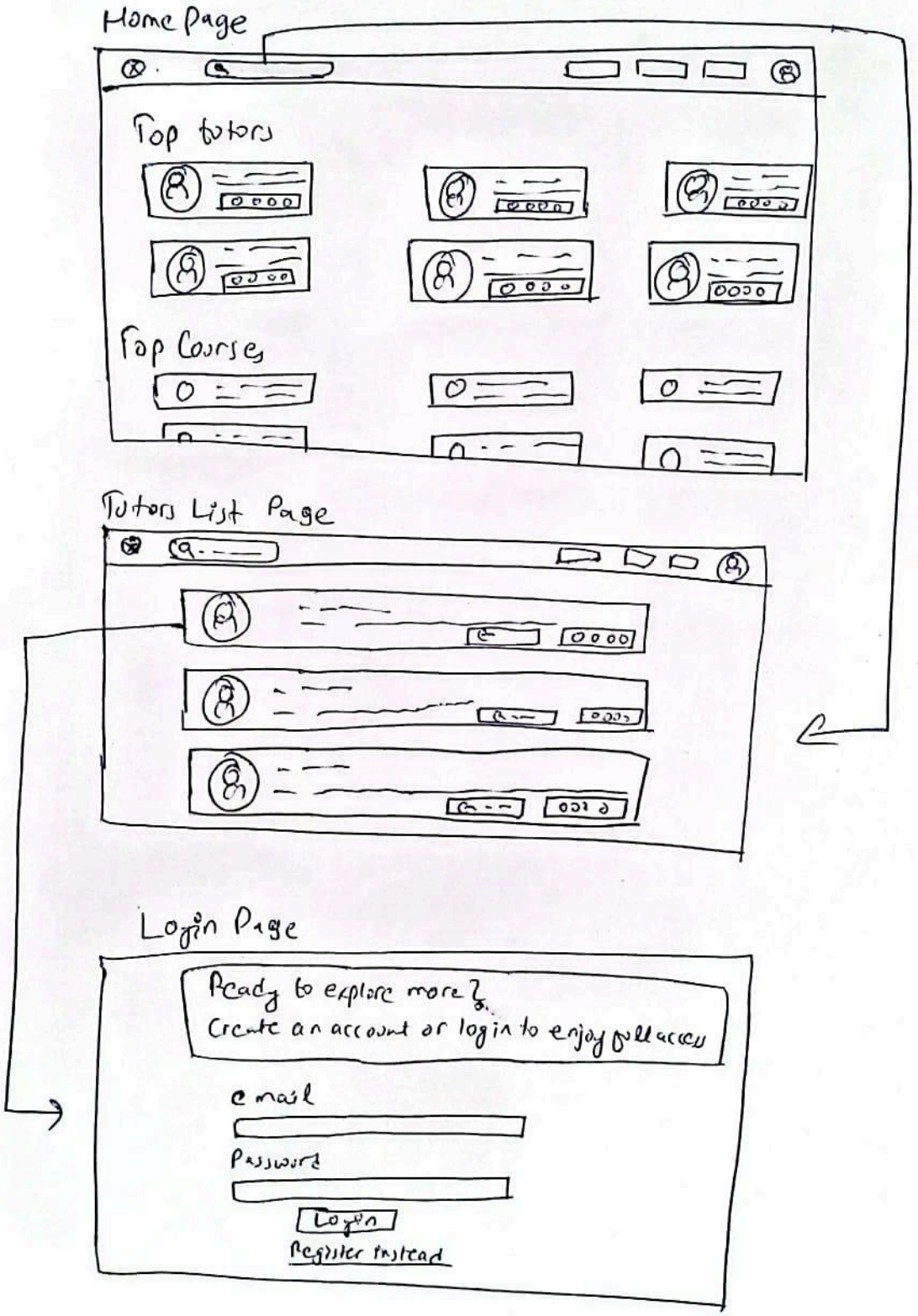
3.2 UI Mockups and Storyboards

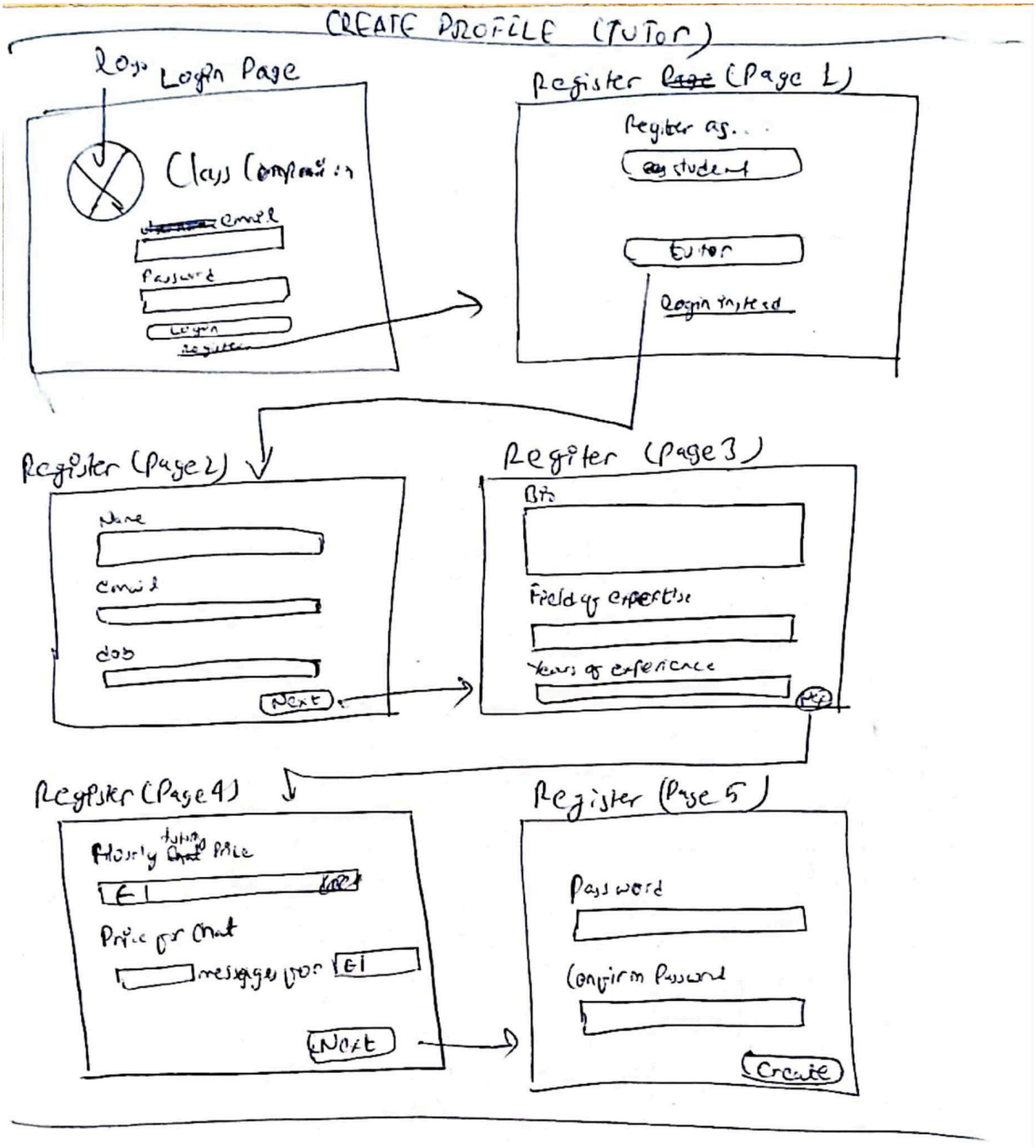


Ask tutors help (student)



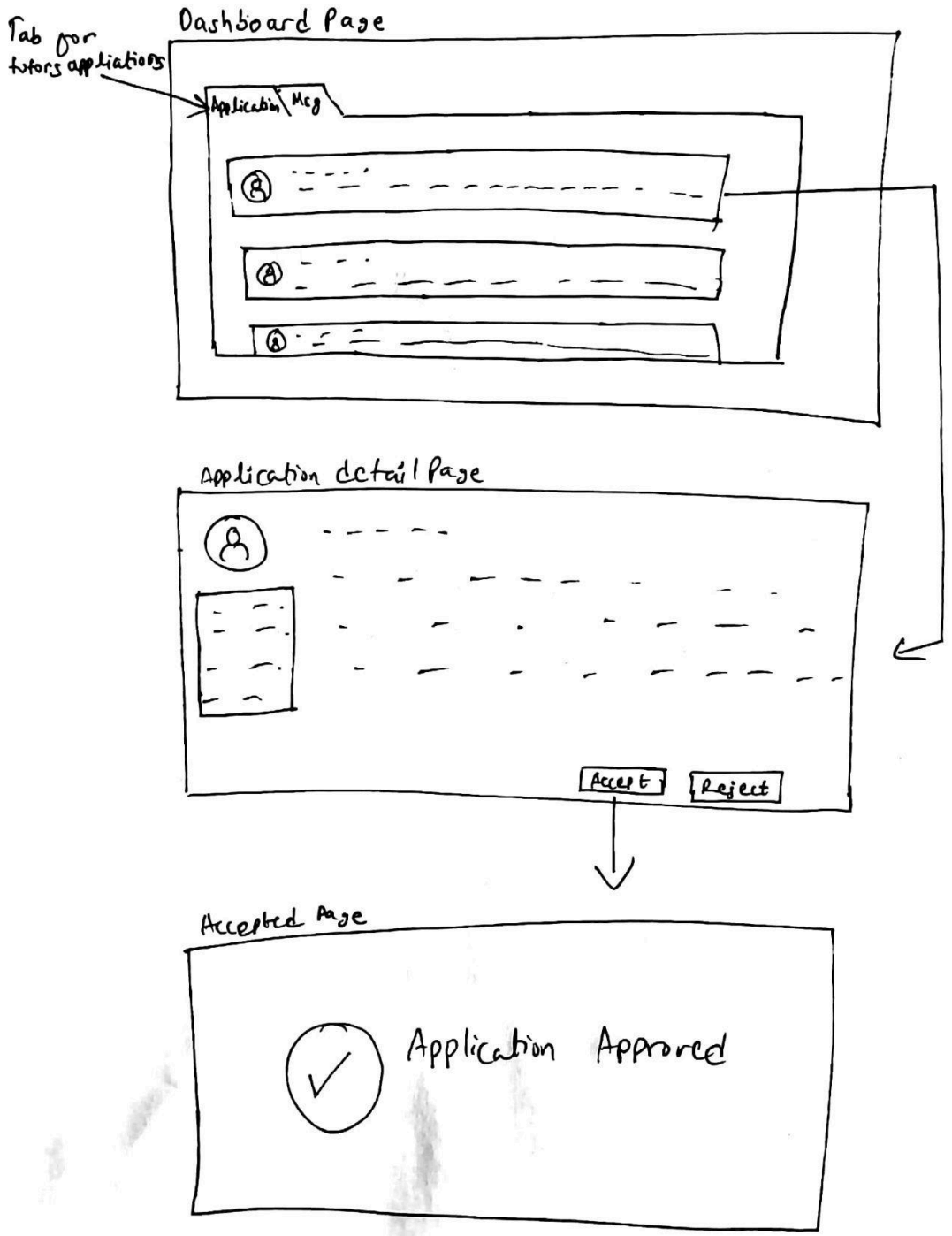
Guest Access (Guest)




CREATE PROFILE (TUTOR)
Login Page
Class
email
Password
Login
Register
Register (Page 1)
Register as...
as student
Tutor
Register (Page 2)
Name
email
dob
Next
Register (Page 3)
Bio
Field of expertise
Years of experience
Register (Page 4)
Hourly tutoring Price
Price per chat
messages per
Next
Register (Page 5)
Password
Confirm Password
Create

Accept Tutor Application (Administrator)

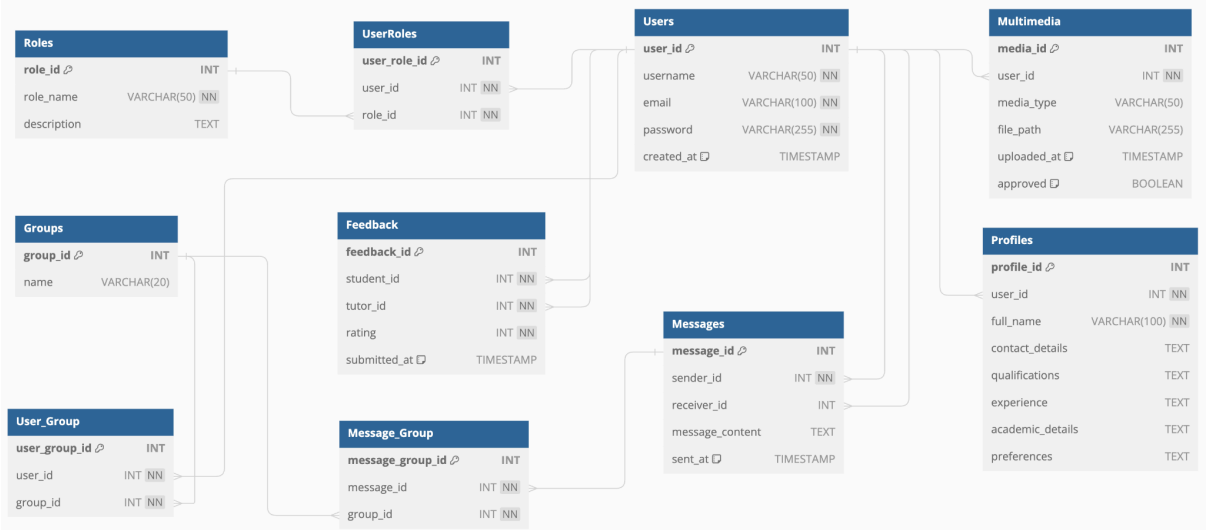
(This will be in a seperate dashbord app)



4. High level Architecture, Database Organization

The database schema includes several main tables to manage users, roles, permissions, profiles, messages, services, feedback, administration, search, and multimedia. Below is a detailed schema that includes RBAC and the additional features.

1. **Users Table**
Stores information about the users in the system.
2. **Roles Table**
Stores the different roles that users can have.
3. **UserRoles Table**
Associates users with roles. A user can have multiple roles.
4. **Profiles Table**
Stores comprehensive information about all registered students and tutors.
5. **Messages Table**
Stores messages exchanged between users.
6. **Feedback Table**
Stores student ratings of tutors and reviews about their tutoring experiences.
7. **Multimedia Table**
Stores information about multimedia files uploaded by users.
8. **Groups Table**
Stores information about groups.
9. **User_Group Table**
Associates users with groups. A user can have multiple groups.
10. **Message_Group Table**
Associates messages with groups.



Media Storage

Multimedia files (e.g., CVs, files, lectures) will be stored in a dedicated file system AWS S3. The database will store metadata and file paths to ensure quick access and management.

Search/Filter Architecture and Implementation

The search functionality will be implemented using SQL with the %LIKE% operator for text searches. Indexed columns will ensure faster search performance.

Search Algorithm Steps:

1. User inputs keywords into the search bar.
2. The system constructs a SQL query using the %LIKE% operator to match keywords with indexed columns.
3. The query is executed, and the results are fetched from the database.
4. Results are formatted and displayed to the user.

APIs

A set of RESTful APIs will be created to handle various functionalities:

1. User Management API
    - Register, login, update profile, delete account
2. Messaging API
    - Send, receive, delete messages
3. Feedback API
    - Submit, view, delete feedback
4. Search API
    - Perform searches based on keywords

Significant Algorithms and Processes

Rating and Ranking Algorithm

Tutors will be ranked based on average ratings, number of reviews, and recency of feedback. A weighted average formula will be used to prioritize recent and highly-rated tutors. To rank tutors effectively, we can use the formula that considers the following factors:

Average Rating: The average rating given by students.
Review Count is the total number of reviews received by the tutor.
Recency Score is calculated based on the timestamps of the reviews, giving more weight to recent reviews.

Here's a proposed weighted average formula:

Tutor Score = ($w1$ × Average Rating) + ($w2$ × Review Count) + ($w3$ × Recency Score)

Where: $w1$, $w2$, $w3$ are the weights assigned to each factor, respectively.

Example Calculation

Assume a tutor has the following metrics:

Average Rating: 4.6
Review Count: 50
Number of Reviews in the Last 30 Days: 10

Let's assume the following example weights:

$w1$ = 0.5
$w2$ = 0.3
$w3$ = 0.2

Combining the factors, we get:

Tutor Score = (0.5 × Average Rating) + (0.3 × Review Count) + (0.2 × Recency Score) = (0.5 × 4.6) + (0.3 × 50) + (0.2 × 10) = 19.3

SQL Implementation

To implement this simplified formula in SQL, we can use a single query to calculate and rank tutors:

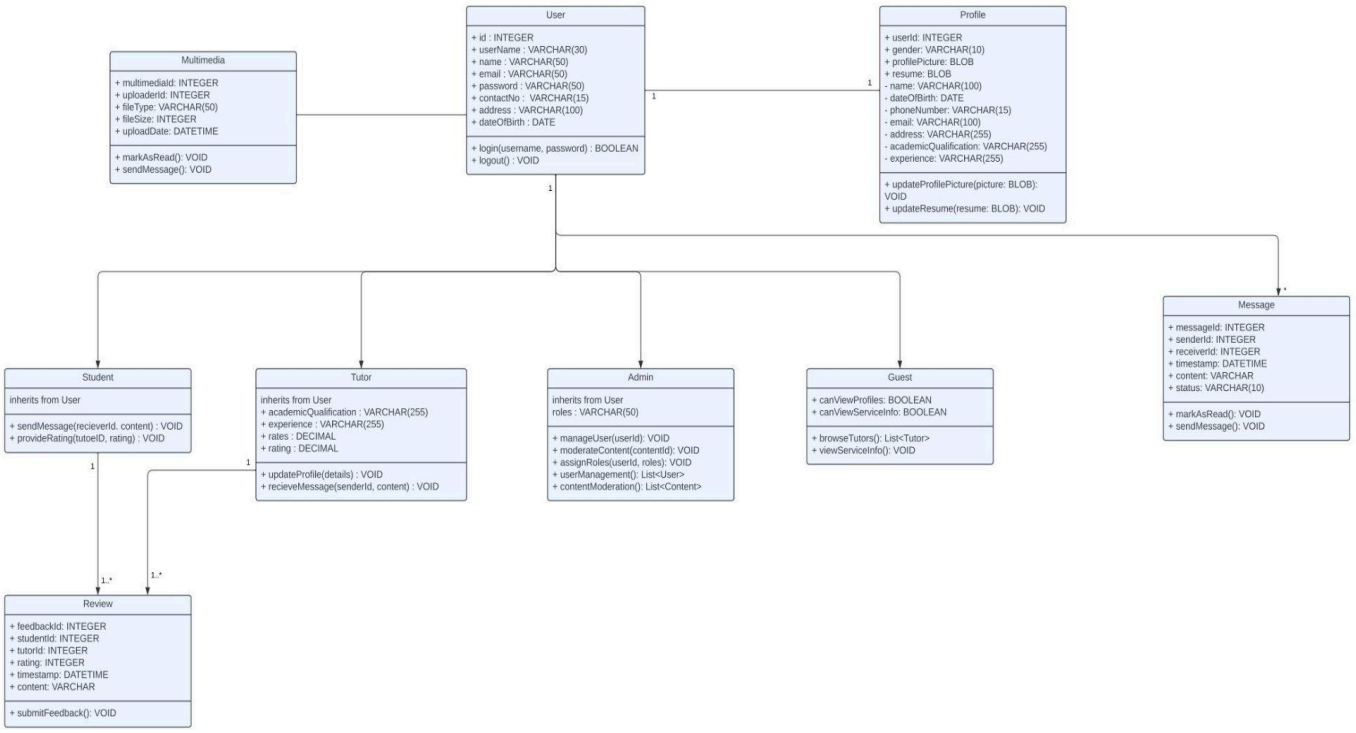
```
SELECT
  tutor_id,
  (0.5 * AVG(rating)) +
  (0.3 * COUNT(feedback_id)) +
  (0.2 * SUM(CASE
    WHEN DATEDIFF(CURRENT_DATE, submitted_at) <= 30 THEN 1
    ELSE 0
  END)) AS tutor_score
FROM
  Feedback
GROUP BY
  tutor_id
ORDER BY
  tutor_score DESC;
```

Software Tools and Frameworks

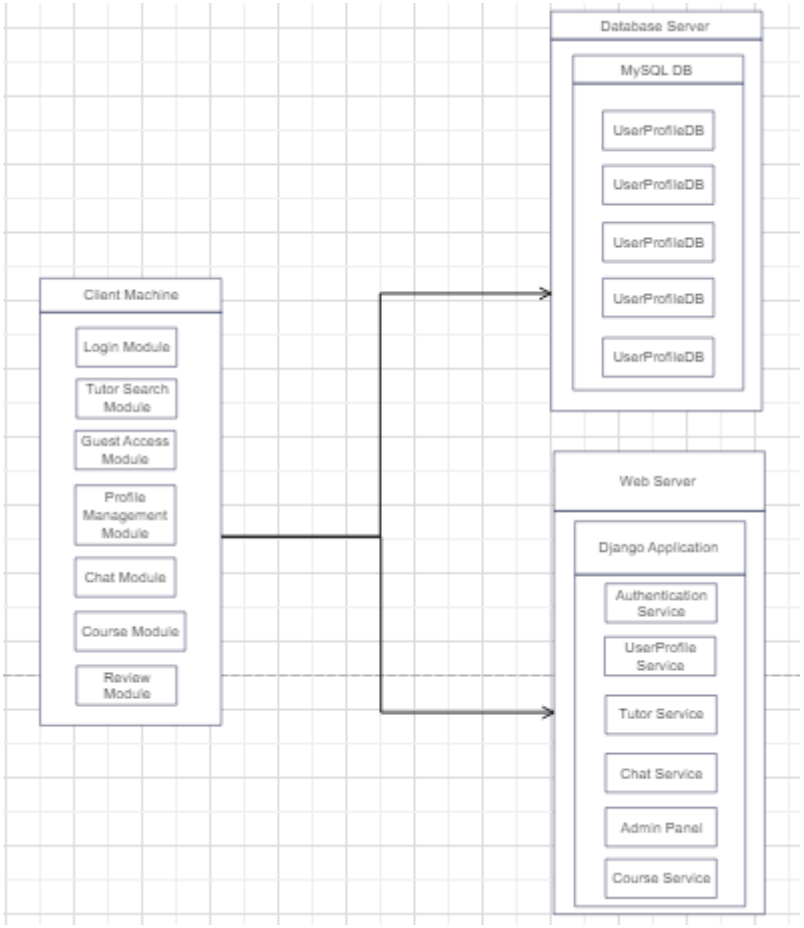
Media Storage: AWS S3 for secure and scalable file storage.

5. High Level UML Diagrams

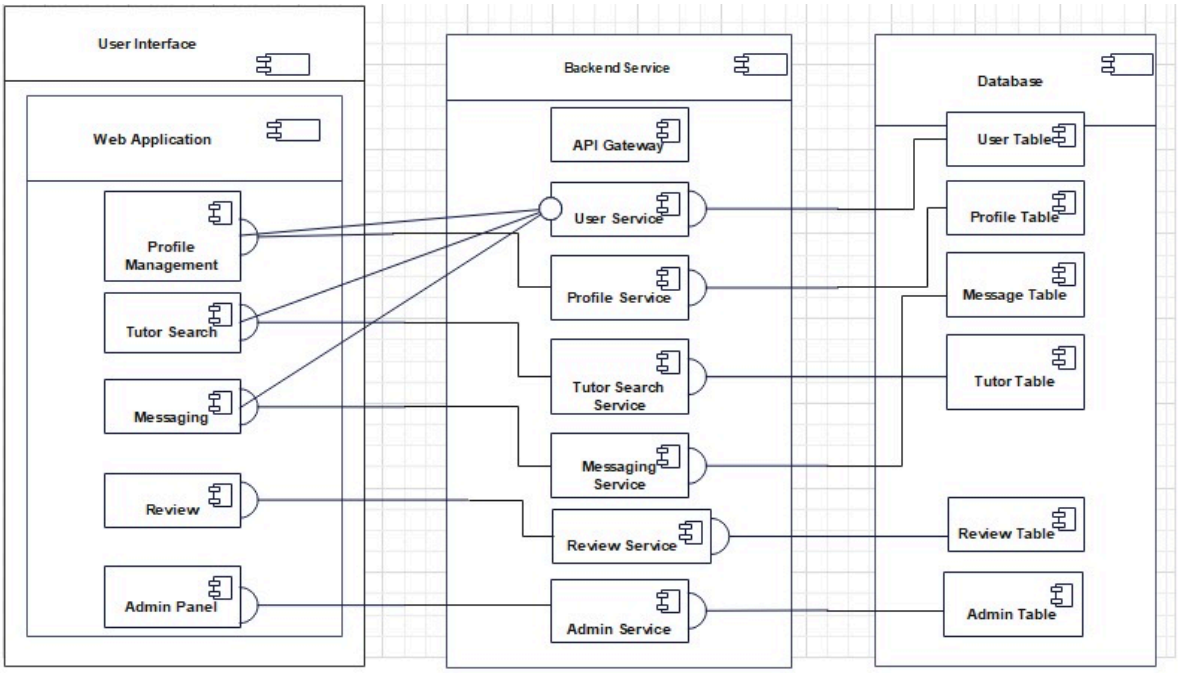
UML class diagrams



Deployment diagram



Component diagram



6. Key Risks for the Project Server Related Issues

Server Related Issues:

Risk: Issues related to AWS or other hosting services, such as unreliable systems or AWS credit issues, could disrupt the project's progress.
Resolution: Maintain a standby AWS account in an unused state to be activated immediately if the main account's credit expires. Additionally, pre-plan alternative hosting options to switch quickly if one fails, ensuring continuous service availability.

Git Related Issues:

Risk: Accidental faulty commits or deletion of significant code portions by team members can lead to application breakdowns and data loss.
Resolution: Create local backups periodically to prevent substantial data loss. Always pay close attention to PRs and merge only if approved. In case of data loss, recover code from backups.

Skill-Related Issues:

Risk: Team members may encounter difficulties in delivering solutions promptly, particularly if they are still learning new technologies or processes.
Resolution: Redistributing tasks under the guidance of front-end or back-end leads helps balance the workload. Providing additional training and resources supports team members in enhancing their skills, ensuring timely project completion.

Team Collaboration Issues:

Risk: Lack of simultaneous collaboration on features requiring both front-end and back-end input can cause bottlenecks and delays.
Resolution: Ensure proper communication through regular meetings and collaborative tools. Re-plan task priorities to keep everyone in sync and avoid delays. Foster a culture of teamwork and shared responsibility.

Software/Hardware Related Issues:

Risk: Breakdown of personal laptops or PCs, or software issues like code editors, compilers, interpreters, or operating system corruption can halt individual contributions and slow down the team's progress.
Resolution: Factor in possible equipment failures when preparing the timeline to ensure deadlines are not affected.

Technical Dependency Issues:

Risk: Compatibility issues with frameworks or tools (e.g., a Python or JS library incompatible with certain browsers or OS) can require significant rework.
Resolution: Regularly test for compatibility issues throughout the development process. Allocate extra time in the timeline for potential changes in libraries or source code adjustments. Have a backup plan for alternative technologies if needed.

Team Member's Health or Personal Issues:

Risk: Significant absenteeism from team members due to health or personal issues can affect the project timeline and delivery.
Resolution: Encourage the team to give extra time to the project if necessary or renegotiate deadlines to accommodate unforeseen circumstances. Maintain a flexible task allocation system to cover for absent members and ensure continuous progress.

Legal Content Dependency Issues:

Risk: Delays in obtaining required permissions from third parties or the university can halt progress on specific features that depend on external data.
Resolution: Continue development using dummy data that closely resembles the required data. Integrate the actual data once permissions are obtained without significant rework. Maintain clear communication with third parties to expedite the approval process.

7. Project management

Strategy

To ensure that the front-end and back-end teams can operate independently while maintaining effective collaboration, we will implement a structured project management strategy. This strategy includes clear task assignment, setting deadlines, and using appropriate tools for tracking progress. The focus is on creating well-defined interfaces and endpoints that both teams agree upon, facilitating seamless integration.

Tools and Methods

1. Task Management Tool: GitHub

- GitHub will be our primary tool for managing tasks. We will use the “issues” feature of Github to track tasks.
- Labels and Milestones: Issues will be organized using labels (e.g., "frontend," "backend," "bug," "enhancement") and milestones to categorize and prioritize tasks.
- Assignees and Deadlines: Each issue will have an assigned person responsible for its completion and a deadline to ensure accountability and timely progress.

2. Regular Meetings

- Daily Chat on a messenger: Instead of daily stand-ups, team members will check in via a group chat on a messaging platform (Whatsapp). This will include updates on what they worked on the previous day, what they plan to work on today, and any blockers they are facing. This approach ensures continuous communication and quick resolution of issues.

- Weekly Syncs: A weekly meeting for both teams to discuss progress, review common interfaces, and ensure alignment on goals and deadlines.

3. Clear Interface Agreements

- API Documentation: We will use tools like Swagger to create comprehensive API documentation that both teams can reference. This ensures that the front-end team knows exactly what data to expect from the back-end and how to interact with it.
- Mock Servers: For the front-end team to work independently, we will set up mock servers that simulate the API responses. This allows the front-end team to develop and test their features without depending on the back-end development being completed.

Master Team Project Summer 2024
ClassCompanion
Milestone 4

Team 1

Yagubkhan Yagubkhanov, Team Lead / Github Master
yagubkhan.yagubkhanov@informatik.hs-fulda.de

Samman Adhikari, Frontend Lead

Shantanu Patil, Backend Lead

Aralikere Yogananda Gunarachana

Mariha Asif

Amber Asad

Date: 15.07.2024

Revision Summary:

Date	Revision	Summary
08.07.2024	1.0	Initial draft created
15.07.2024	1.1	Made changes based on instructor's feedback

Content and structure for Milestone 4

1. Product summary
2. Usability test plan
3. QA test plan
4. Code review
5. Self-check on best practices for security
6. Self-check: Adherence to original Non-functional specs – performed by team leads

1. Product Summary

Product Name

ClassCompanion

Product Overview

ClassCompanion is a comprehensive platform designed to facilitate seamless interaction between students and tutors. By integrating essential features such as user registration, profile management, search functionality, and real-time communication, ClassCompanion aims to enhance the educational experience by providing an accessible and efficient tutoring service.

Major Committed Functions

User Registration and Login

Students and tutors must register using their university email address, with email validation to ensure eligibility, and securely log in and log out with the option to register as a student or tutor via a checkbox.

Profile Management

Users (students and tutors) can manage their profiles, update personal information, add skills, and upload profile pictures, with students restricted from searching for other students based on skills or subjects.

Tutor Search Functionality

Students can search for tutors based on subjects, keywords, experience, and rating with search results displaying a list of relevant tutors based on the search criteria.

Live Chat Feature

Students can initiate live chat sessions with tutors, with chat functionality supporting text messages, file uploads (such as documents or images), and other multimedia content, and offline messages delivered to tutors' inboxes for later viewing.

Inbox Management

Both tutors and students have access to their respective inboxes, displaying messages received from other users, including chat messages and notifications, with users able to mark messages as read, reply, and delete messages as needed.

Admin Panel

There are two cases where a user needs admin's approval. One is when the user signs up and the other is when the user tries to change media like his/her profile picture or cv. This is to make sure that no media that violates our policy can be public. With the admin panel, the moderator/admin will be able to do exactly these things.

Unique Features

Group Chat Functionality

Facilitates collaborative learning by enabling tutors to engage with multiple students at once, promoting group study sessions and discussions.

Appointment Booking System

Facilitates personalised learning by allowing students to easily view tutor availability and book one-on-one appointments. This feature ensures effective communication and provides students with the opportunity to address their questions and concerns directly with their chosen tutors, thereby enhancing the overall learning experience.

URL to Access the Product:

http://35.158.64.195/login

2. Usability Test Plan for ClassCompanion

This document describes a test plan for conducting a usability test during the development of ClassCompanion.

Selected Function for Testing

Chat Functionality

Test Objectives

The objective of this usability test is to evaluate the chat functionality of ClassCompanion. This includes assessing the ease of initiating a chat, sending messages, and maintaining a conversation between students and tutors. The test aims to measure user satisfaction, identify any usability issues, and ensure that the chat feature meets user expectations for an educational platform.

Test Background and Setup

System Setup: The test will be conducted using the deployed version of ClassCompanion on a standard laptop or mobile phone with internet access. The URL for accessing the system will be provided to the participants.

Starting Point: Participants will begin at the ClassCompanion homepage, where they will log in using pre-registered accounts. They will then navigate to the chat feature.

Intended Users: The intended users for this test are students and tutors. The participants selected for this test include students pursuing different levels of higher education and a teaching assistant to ensure a diverse range of feedback.

URL of the System to be Tested: http://35.158.64.195/login (this URL will be provided to the participants prior to testing).

What is to be Measured: For this test, the primary focus will be on user satisfaction with the chat functionality. This will be evaluated through a Likert scale questionnaire, capturing user feedback on various aspects of the chat experience, including ease of use, clarity, and responsiveness.

Usability Task Description

Participants will be given the following instructions for the usability test:

1. **Task 1:** Imagine you need help with a specific academic topic (e.g., calculus homework, research paper, or lecture preparation). Initiate a chat with a tutor on ClassCompanion.
2. **Task 2:** Send a message to the tutor describing your problem or topic of interest.
3. **Task 3:** Engage in a conversation with the tutor for a few exchanges, noting any difficulties or issues encountered.

Effectiveness Measurement: Effectiveness will be measured by the participants' ability to successfully initiate a chat, send messages, and maintain a conversation without encountering major issues.

Efficiency Measurement: Efficiency will be measured by the time taken to complete each task and the number of steps required to perform each action within the chat functionality.

Likert Scale Questions for User Satisfaction:

Participants will use a Likert scale to assess their satisfaction with the chat functionality in ClassCompanion after completing the usability tasks. They will rate their experience based on the following criteria:

1. Overall Experience: How would you rate your overall experience with the chat functionality in ClassCompanion?
    - 1 (Very Poor)
    - 2 (Poor)
    - 3 (Neutral)
    - 4 (Good)
    - 5 (Very Good)
2. Interface Usability: How simple and clear was the chat interface to use?
    - 1 (Very Complicated)
    - 2 (Complicated)
    - 3 (Neutral)
    - 4 (Simple)
    - 5 (Very Simple)
3. Responsiveness: How responsive was the chat functionality during your interaction?
    - 1 (Very Unresponsive)
    - 2 (Unresponsive)
    - 3 (Neutral)
    - 4 (Responsive)
    - 5 (Very Responsive)

These questions are designed to gather insights into user satisfaction regarding the usability, clarity, and responsiveness of the chat feature in ClassCompanion.

Test Participant Details

Test Participant 1

Screening & Pre-test Questions:

- **Q1:** How old are you? - **A1:** 22
- **Q2:** What is the highest level of education you've completed? - **A2:** Bachelor's Degree
- **Q3:** What is your current occupation? - **A3:** Student (Pursuing Master's)
- **Q4:** Which degree program are you enrolled in? - **A4:** Global Software Development
- **Q5:** On a scale of 1 to 5, how would you rate your level of confidence in using your laptop/PC for online communication? - **A5:** 5
- **Q6:** When was the last time you used an online chat application for educational purposes? - **A6:** Last month

Likert Scale Questions Responses:

Criteria	Response
Criteria 1	5/5
Criteria 2	5/5
Criteria 3	4/5

Post Test Questions:

- **Q1:** How was your overall experience when using the chat functionality? - **A1:** Good
- **Q2:** How simple and clear was the chat interface? - **A2:** It is very simple and clear with instructions.
- **Q3:** Did you encounter any issues during the chat? - **A3:** No, it worked without any issues.
- **Q4:** How was your experience with the responsiveness of the chat? - **A4:** Simple and clear.
- **Q5:** How was your overall experience using ClassCompanion? - **A5:** It was very easy to use, and I faced no issues during the chat process.

Test Participant 2

Screening & Pre-test Questions:

- **Q1:** How old are you? - **A1:** 24
- **Q2:** What is the highest level of education you've completed? - **A2:** Master's Degree
- **Q3:** What is your current occupation? - **A3:** Research Assistant
- **Q4:** Which degree program are you enrolled in? - **A4:** Global Software Development
- **Q5:** On a scale of 1 to 5, how would you rate your level of confidence in using your laptop/PC for online communication? - **A5:** 4
- **Q6:** When was the last time you used an online chat application for educational purposes? - **A6:** Two months ago

Likert Scale Questions Responses:

Criteria	Response
Criteria 1	5/5
Criteria 2	5/5
Criteria 3	5/5

Post Test Questions:

- **Q1:** How was your overall experience when using the chat functionality? - **A1:** Very Good
- **Q2:** How simple and clear was the chat interface? - **A2:** It is straightforward and easy to use.
- **Q3:** Did you encounter any issues during the chat? - **A3:** No, it was smooth.
- **Q4:** How was your experience with the responsiveness of the chat? - **A4:** Very responsive.
- **Q5:** How was your overall experience using ClassCompanion? - **A5:** It was user-friendly and efficient for my needs.

Test Participant 3

Screening & Pre-test Questions:

- **Q1:** How old are you? - **A1:** 26
- **Q2:** What is the highest level of education you've completed? - **A2:** PhD Candidate
- **Q3:** What is your current occupation? - **A3:** Teaching Assistant
- **Q4:** Which program are you a teaching assistant in? - **A4:** Msc. Chemical Science
- **Q5:** On a scale of 1 to 5, how would you rate your level of confidence in using your laptop/PC for online communication? - **A5:** 5
- **Q6:** When was the last time you used an online chat application for educational purposes? - **A6:** Last week

Likert Scale Questions Responses:

Criteria	Response

Criteria 1	5/5
Criteria 2	5/5
Criteria 3	5/5

Post Test Questions:

- **Q1:** How was your overall experience when using the chat functionality? - **A1:** Excellent
- **Q2:** How simple and clear was the chat interface? - **A2:** It is very intuitive and user-friendly.
- **Q3:** Did you encounter any issues during the chat? - **A3:** No, it worked flawlessly.
- **Q4:** How was your experience with the responsiveness of the chat? - **A4:** Very quick and reliable.
- **Q5:** How was your overall experience using ClassCompanion? - **A5:** It was seamless and highly effective for my purposes.

Test Participant 4

Screening & Pre-test Questions:

- **Q1:** How old are you? - **A1:** 29
- **Q2:** What is the highest level of education you've completed? - **A2:** Bachelor's Degree
- **Q3:** What is your current occupation? - **A3:** Student
- **Q4:** Which degree program are you enrolled in? - **A4:** M.A. Human Rights Studies in Politics, Law and Society
- **Q5:** On a scale of 1 to 5, how would you rate your level of confidence in using your laptop/PC for online communication? - **A5:** 5
- **Q6:** When was the last time you used an online chat application for educational purposes? - **A6:** Last week

Likert Scale Questions Responses:

Criteria	**Response**

Criteria 1	4/5
Criteria 2	5/5
Criteria 3	5/5

Post Test Questions:

- **Q1:** How was your overall experience when using the chat functionality? - **A1:** Good
- **Q2:** How simple and clear was the chat interface? - **A2:** It was simple and clear. But the UI seemed a bit off. Some choice of colours made the interface look a bit unaesthetic.
- **Q3:** Did you encounter any issues during the chat? - **A3:** No, nothing during chat.
- **Q4:** How was your experience with the responsiveness of the chat? - **A4:** It seemed very quick and responsive.
- **Q5:** How was your overall experience using ClassCompanion? - **A5:** It was great.

Test Participant 5

Screening & Pre-test Questions:

- **Q1:** How old are you? - **A1:** 23
- **Q2:** What is the highest level of education you've completed? - **A2:** Bachelor's Degree
- **Q3:** What is your current occupation? - **A3:** Student
- **Q4:** Which degree program are you enrolled in? - **A4:** M.A. Human Rights Studies in Politics, Law and Society
- **Q5:** On a scale of 1 to 5, how would you rate your level of confidence in using your laptop/PC for online communication? - **A5:** 5
- **Q6:** When was the last time you used an online chat application for educational purposes? - **A6:** Last week

Likert Scale Questions Responses:

Criteria	**Response**
Criteria 1	4/5

Criteria 2	5/5
Criteria 3	5/5

Post Test Questions:

- **Q1:** How was your overall experience when using the chat functionality? - **A1:** Good
- **Q2:** How simple and clear was the chat interface? - **A2:** It was simple and clear. But the UI seemed a bit off. Some choice of colours made the interface look a bit unaesthetic.
- **Q3:** Did you encounter any issues during the chat? - **A3:** No, nothing during chat.
- **Q4:** How was your experience with the responsiveness of the chat? - **A4:** It seemed very quick and responsive.
- **Q5:** How was your overall experience using ClassCompanion? - **A5:** It was great.

Participants Feedback:

Test Participant 1

- "The chat feature was really easy to use. I liked how simple the interface was, no issues at all."

Test Participant 2

- "Loved the responsiveness of the chat. Everything was very smooth and straightforward. However, I found the message send button a bit too small."

Test Participant 3

- "The chat functionality is excellent! Very intuitive and user-friendly. Had no problems at all. It would be nice to have an option to change the theme."

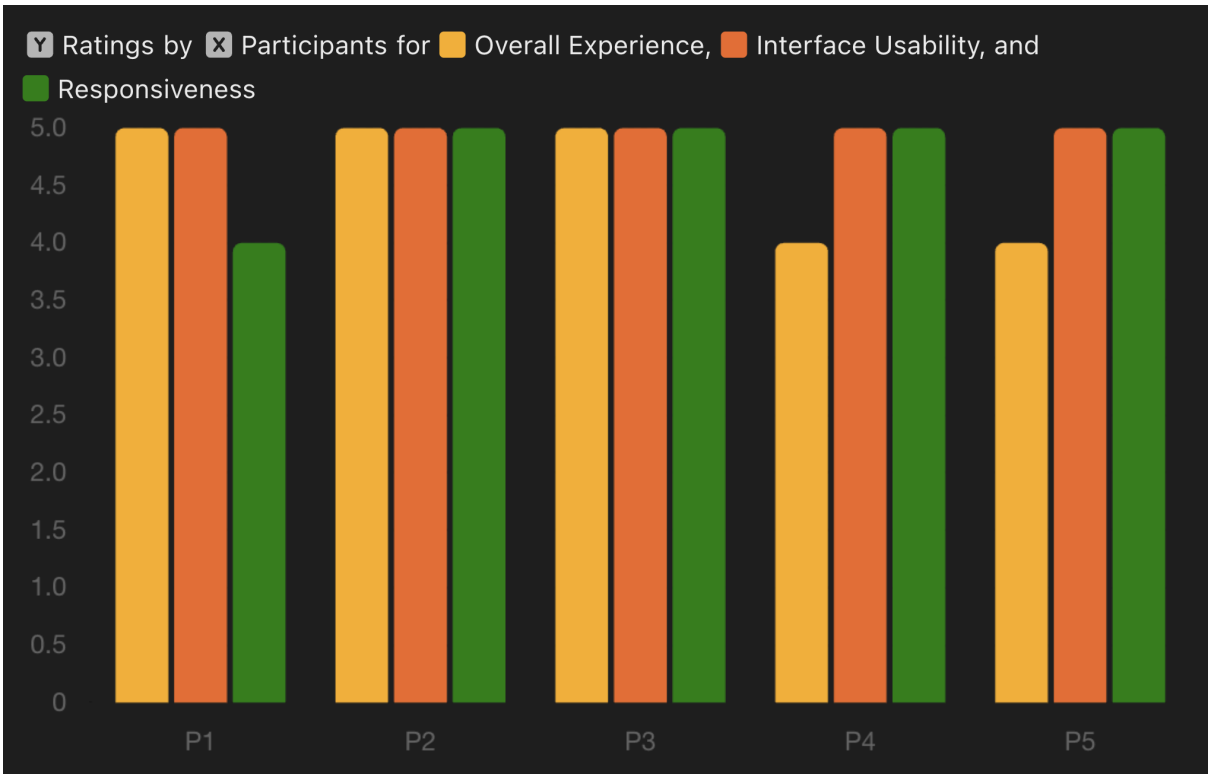
Test Participant 4

- "It worked great overall, but I felt the colors in the interface could be improved a bit. Also, the font size was a little small, making it hard to read."

Test Participant 5

- "Good experience using the chat, but the UI could look a bit better. Functionality-wise, it was perfect. I noticed that the chat window did not auto-scroll to the latest message, which was slightly annoying. Adding emoji support and a typing indicator would enhance the user experience."

Participants Feedback Bar Chart



3. QA Test Plan:

Selected Function for Testing:

Chat Functionality

Test Objectives

The objective of this QA test plan is to verify the functionality, usability, and performance of the chat feature in ClassCompanion. This includes testing message sending, receiving, real-time updates, and overall reliability across different browsers.

HW and SW Setup

- **Hardware Setup:** Standard laptops/desktops with internet connectivity.
- **Software Setup:** Latest versions of Chrome and Firefox browsers.
- **URL:** http://35.158.64.195/login (to be provided to QA testers).

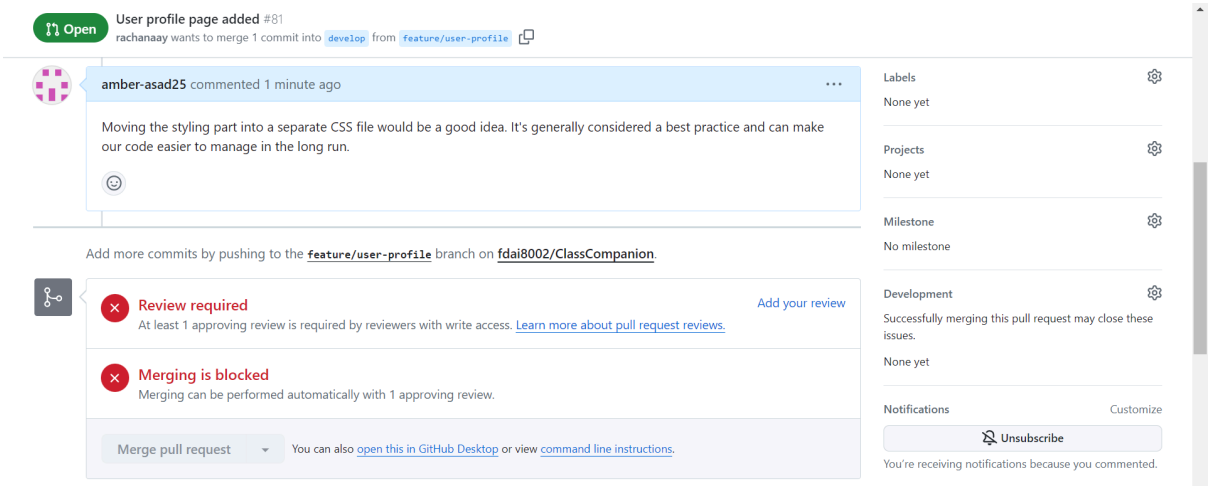
QA Test Plan

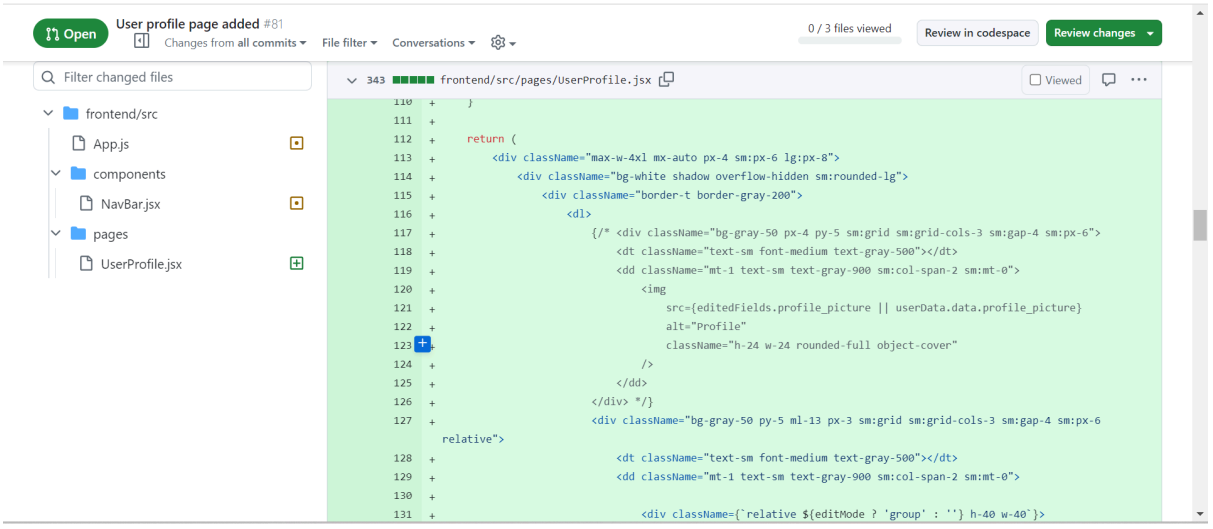
Test #	Test Title	Test Description	Test Input	Expected Correct Output	Test Results (Pass/Fail)
1	Send Message	Test sending a message from student to tutor.	Type message in chat input field	Message is sent and received by tutor	PASS (Chrome v126.0.6478.127), PASS (Firefox) MacOS 14.0
2	Receive Message	Test receiving a message from tutor to student.	Tutor sends message to student	Message is received promptly by student	PASS (Chrome v126.0.6478.127), PASS (Firefox v126.0.1) MacOS 14.0
3	Real-time Updates	Test real-time updates when both users are online.	Send messages simultaneously	Messages appear instantly for both users	PASS (Chrome v126.0.6478.127), PASS (Firefox v126.0.1) MacOS 14.0
4	Offline Message Delivery	Test message delivery when the recipient is offline.	Send message to offline tutor	Message is delivered upon tutor login	PASS (Chrome v126.0.6478.127), PASS (Firefox v126.0.1) MacOS 14.0

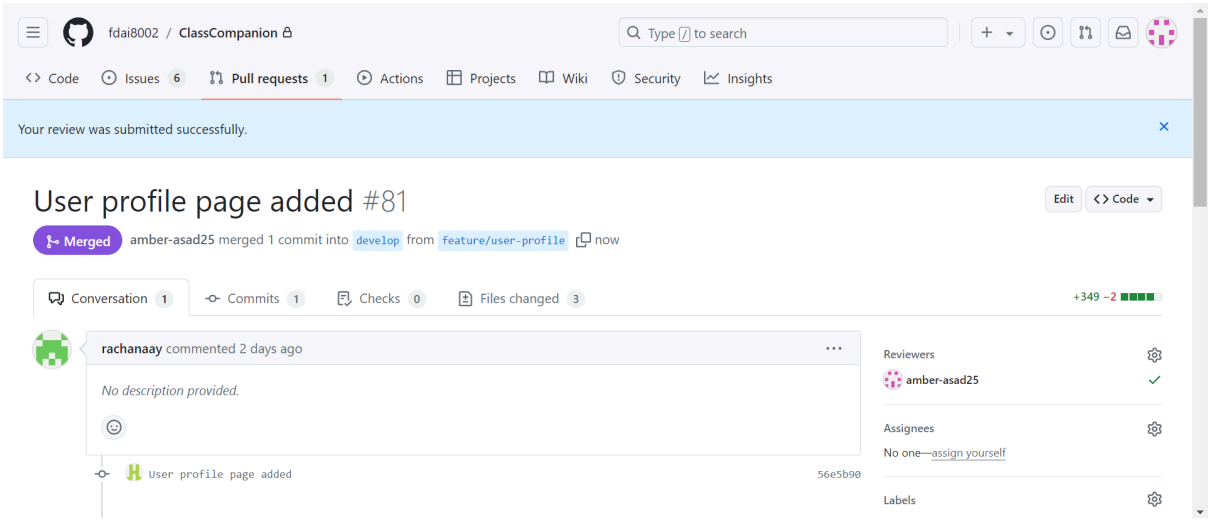
4. Code Review:

Amber Asad code review:

Aralikere Yogananda Gunarachana code reviewed by Amber Asad (https://github.com/fdai8002/ClassCcompanion/pull/81):

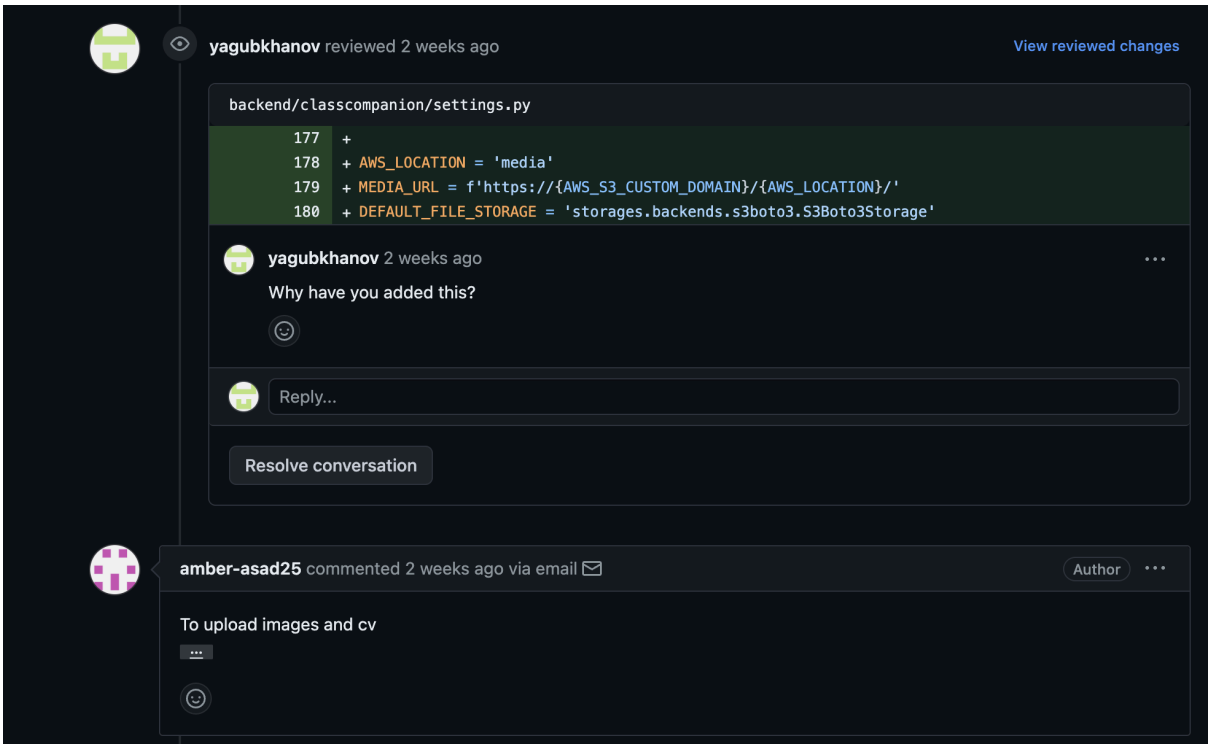




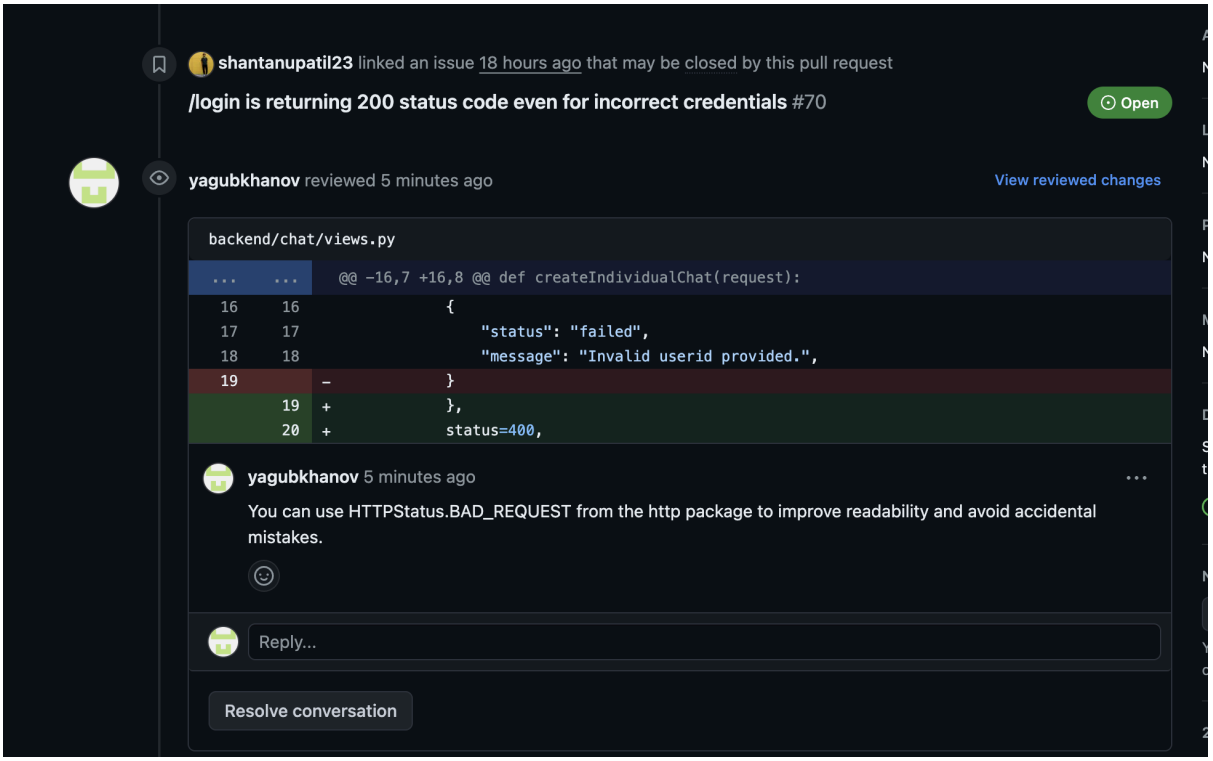


Yagubkhan Yagubkhanov code review:

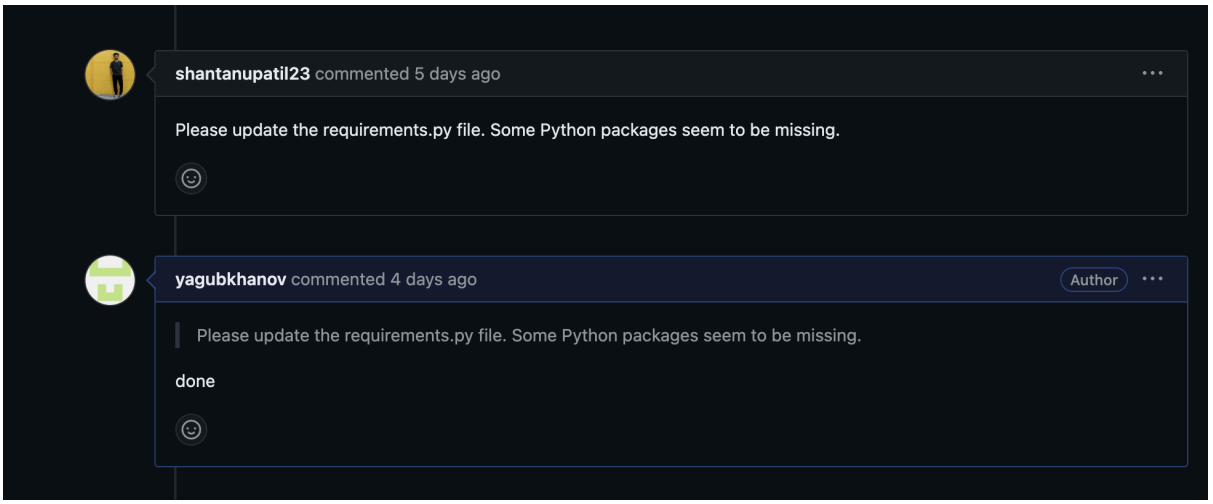
https://github.com/fdai8002/ClassCompanion/pull/45:



https://github.com/fdai8002/ClassCompanion/pull/78:

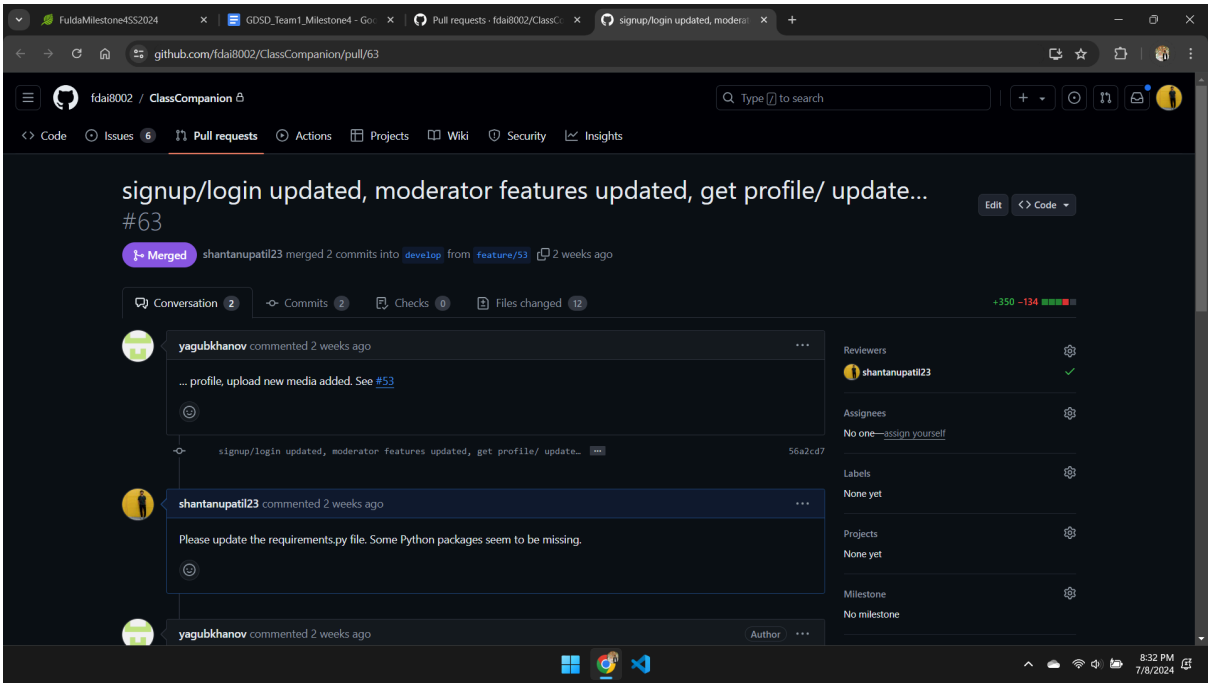


https://github.com/fdai8002/ClassCompanion/pull/63:

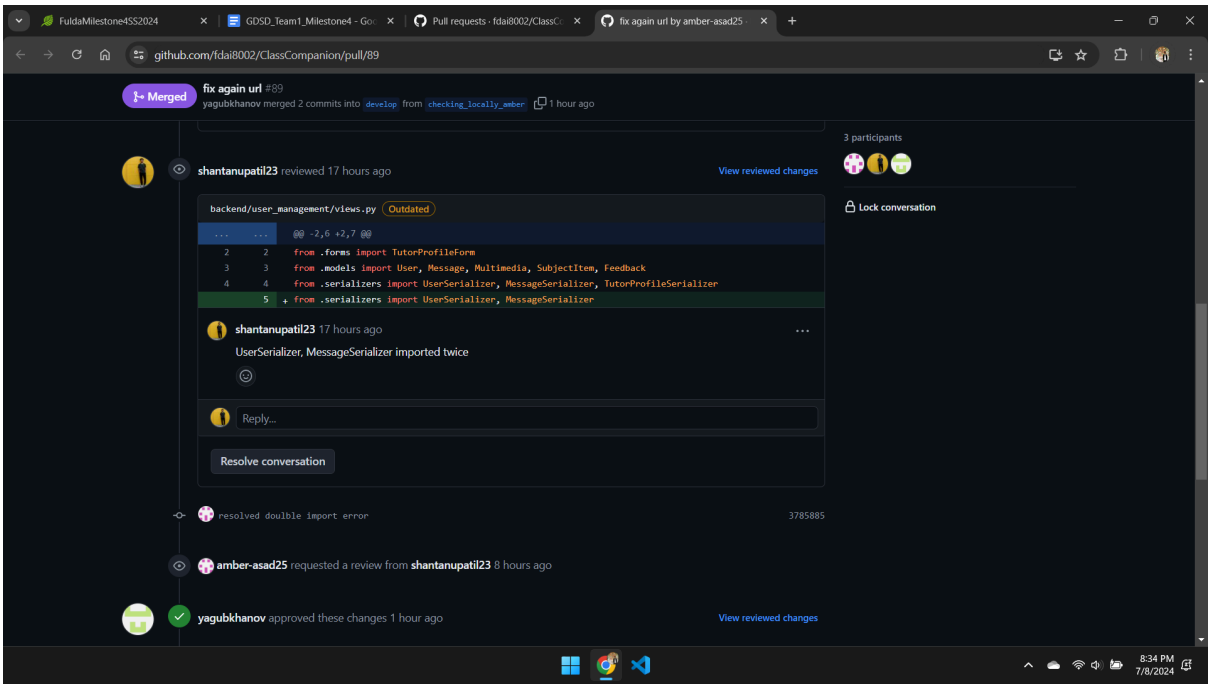


Shantanu Patil code review:

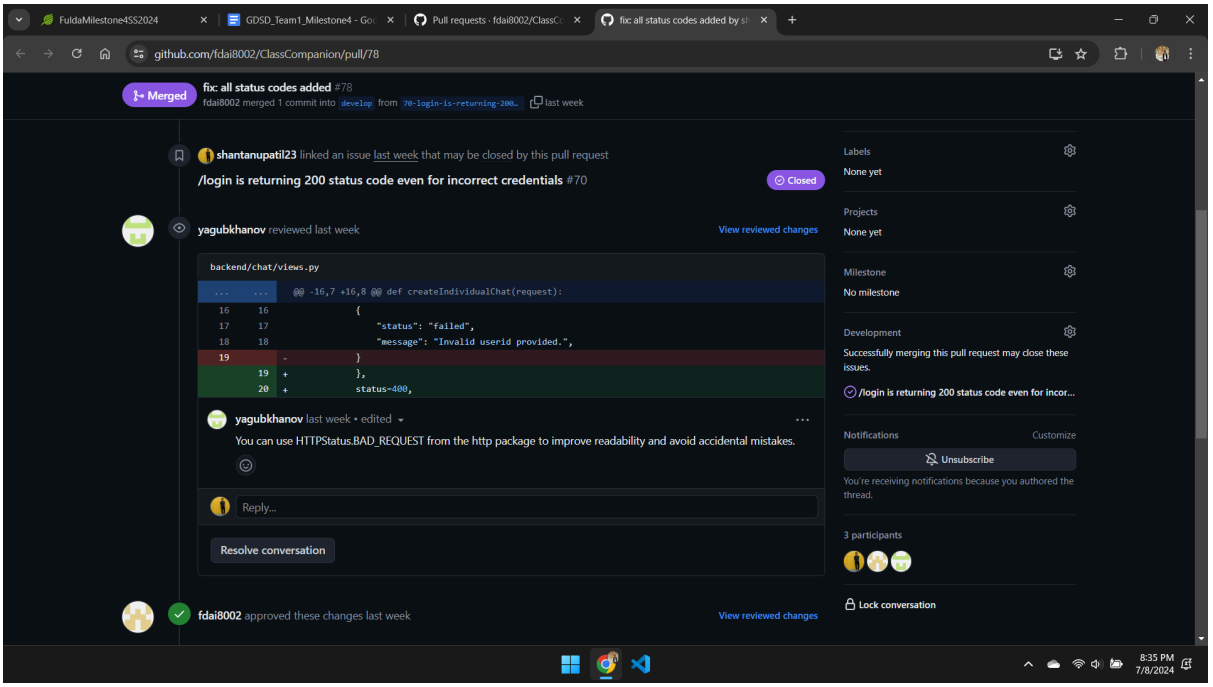
https://github.com/fdai8002/ClassCompanion/pull/63:



https://github.com/fdai8002/ClassCompanion/pull/89:

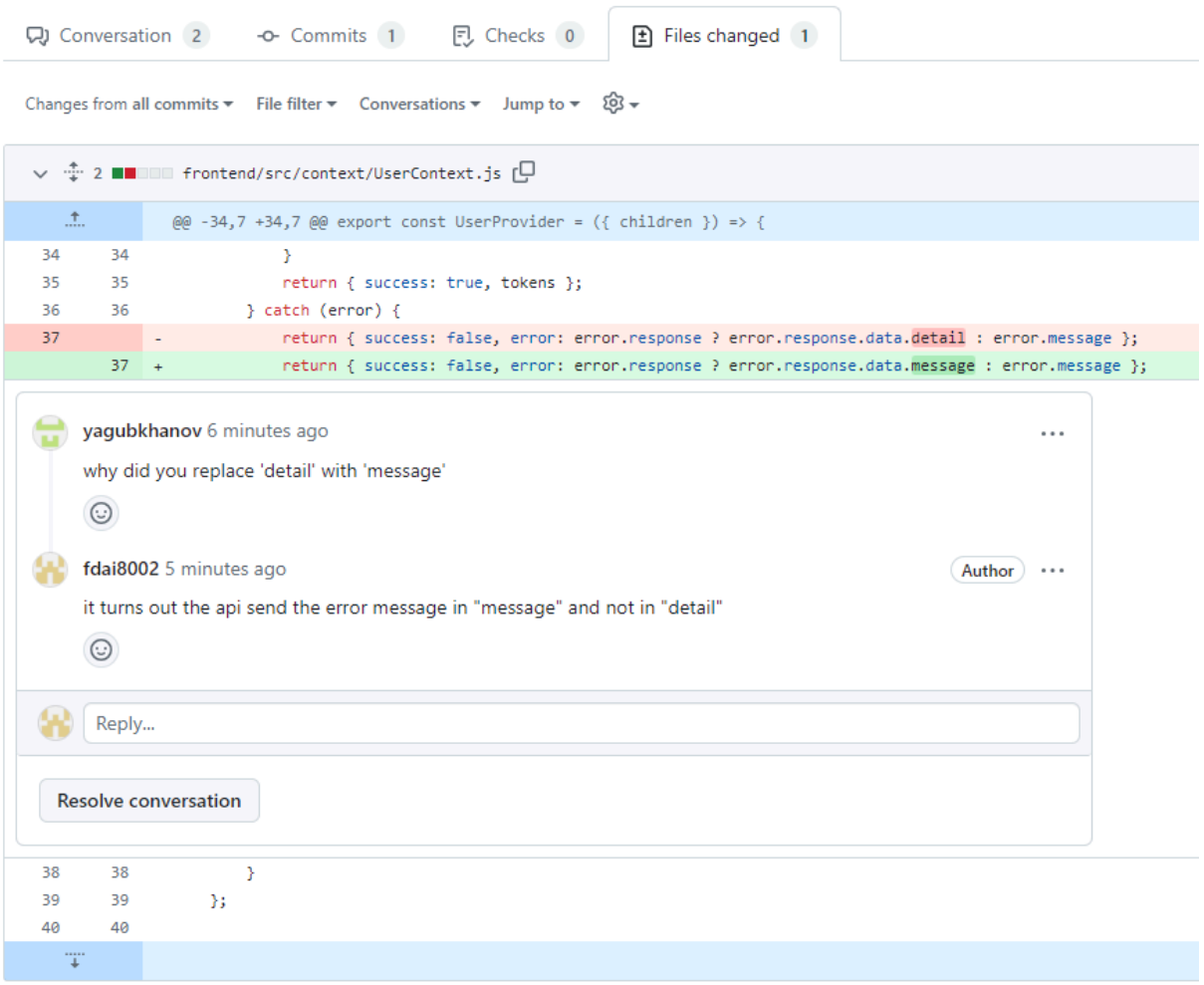


https://github.com/fdai8002/ClassCompanion/pull/78:

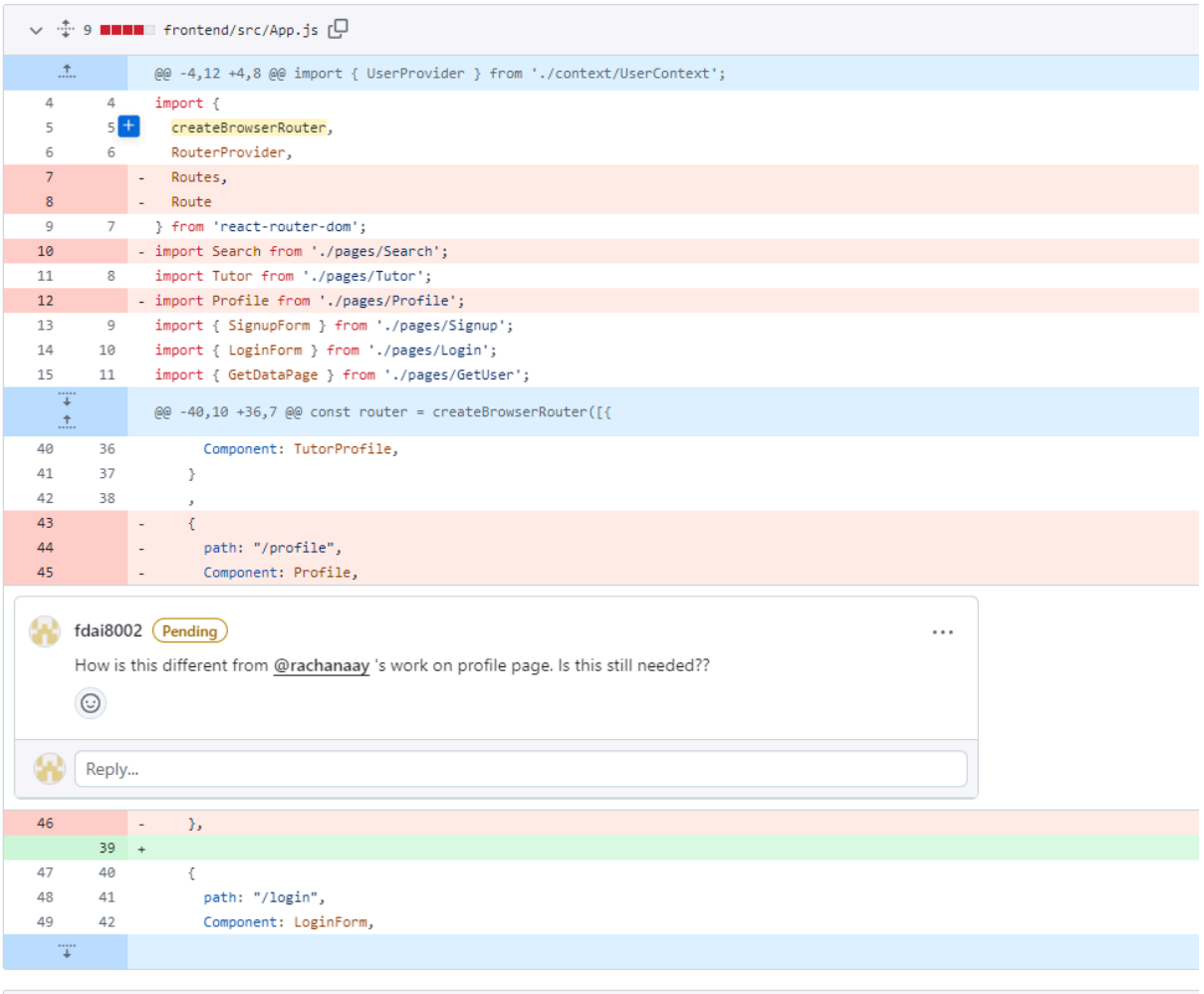


Samman Adhikari (fdai8002) code review:

Samman's PR reviewed by Yagub (https://github.com/fdai8002/ClassCompanion/pull/83)



Amber's PR reviewed by Samman (https://github.com/fdai8002/ClassCompanion/pull/77)



Shantanu’s PR reviewed by Samman (https://github.com/fdai8002/ClassCompanion/pull/84)

d files

nagement

_view.py

backend/chat/views.py

```
@@ -3,6 +3,7 @@
3  3  from rest_framework.decorators import api_view
4  4  from rest_framework_simplejwt.authentication import JWTAuthentication
5  5  from .models import ChatMessage, ChatGroup, User
   6  + from user_management.models import Multimedia
6  7
7  8
8  9  @api_view(["GET"])
@@ -106,8 +107,14 @@ def chatLobby(request):
106 107              chat_type = chat_group.chat_type
107 108
108 109              if chat_type == "personal":
109     -                name = chat_group.users.exclude(id=user.id).first().username
110     -                picture = None
    110 +                sender = chat_group.users.exclude(id=user.id).first()
    111 +                name = sender.username
    112 +                try:
    113 +                    picture = Multimedia.objects.get(
    114 +                        user=sender, media_type="profile_picture"
```

fdai8002 5 minutes ago

what key will this be send in??

fdai8002 1 minute ago

This will cause error because you are returning a database object.

Reply...

Resolve conversation

```
    115 +                )
    116 +            except Exception:
```

Aralikere Yogananda Gunarachana (rachanaay) Code Reviews:

Reviewed Amber Asad's code

rachanaay reviewed 2 minutes ago — View reviewed changes

frontend/src/pages/Tutor.jsx

```
119  +    setError(null); // Clear any previous errors
120  +    try {
121  +      const response = await axios.get(
122  +        `http://localhost:8000/search?query=${searchTerm}`
```

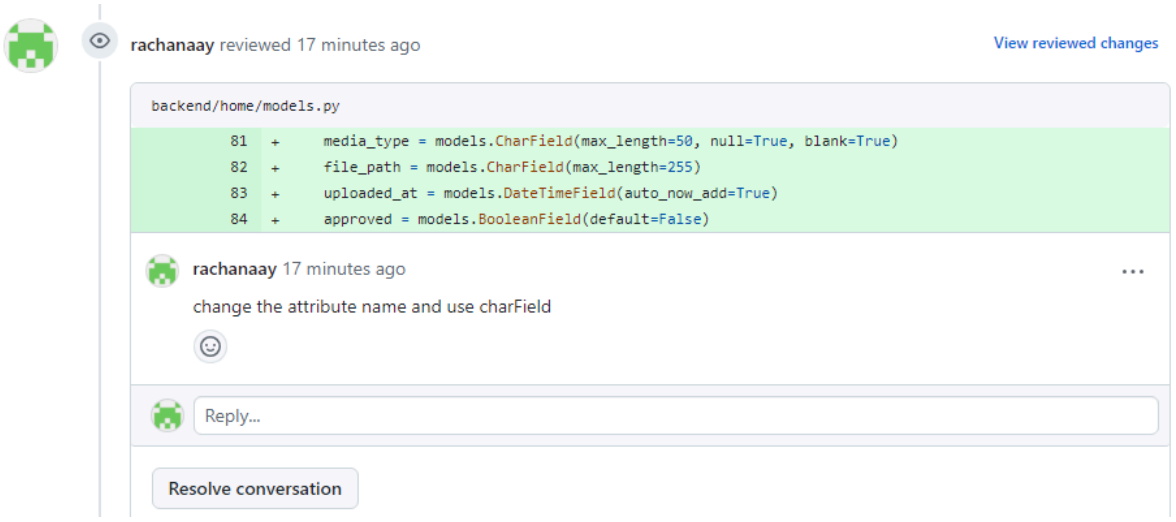
rachanaay 2 minutes ago

Update the Axios request URL to use process.env.REACT_APP_WS_URL

Reply...

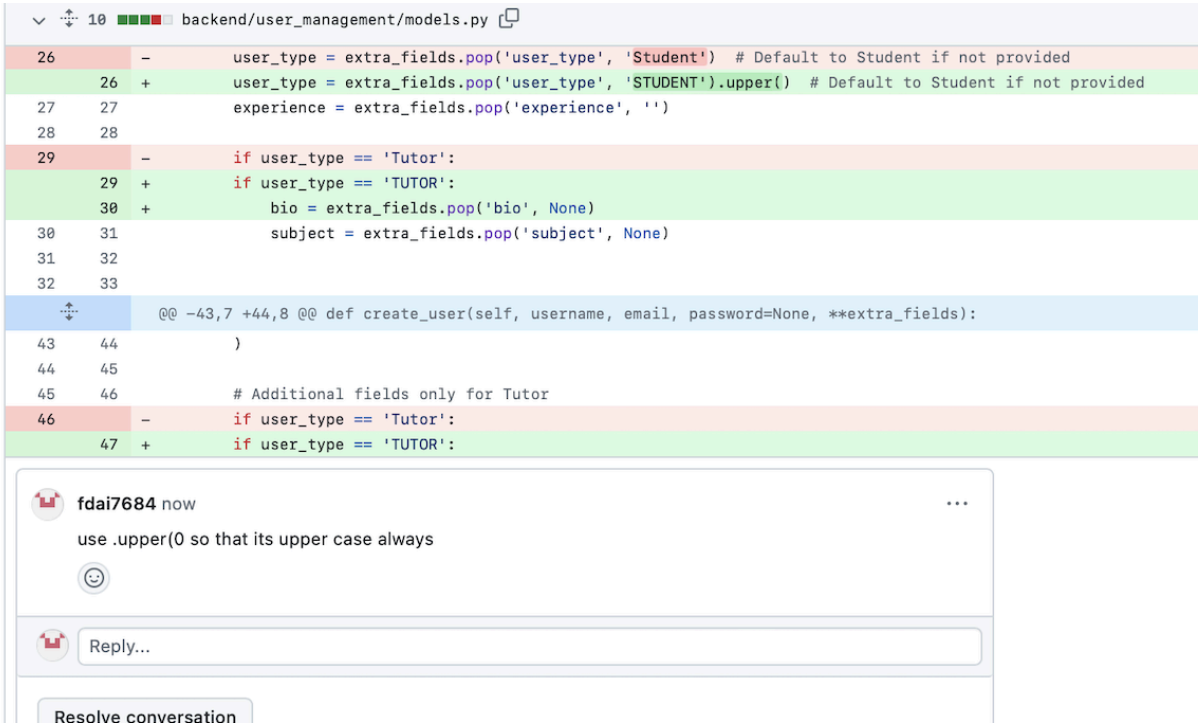
Resolve conversation

Reviewed Yagub's Code



Mariha Asif's (fdai7684) Code Reviews:

Reviewed Shantanu's code



Reviewed Yagub's code



Yagub reviewed Mariha's code



5. Self-check on Best Practices for Security

1. User Authentication and Authorization

• Strong Password Policies: Enforced complex passwords and encourage regular updates.

• Role-Based Access Control (RBAC): Restrict access based on user roles to minimize the risk of unauthorized access.

2. Data Encryption

• Transport Layer Security (TLS): Used HTTPS to encrypt data in transit between clients and servers.

• End-to-End Encryption (E2EE): Implemented E2EE to ensure that only the communicating users can read the messages.

3. Secure Communication Channels

• WebSockets Security: using WebSockets for real-time communication, ensure they are secured with WSS (WebSocket Secure).

4. Server Security

• Regular Software Updates: all server software and dependencies are up to date to mitigate vulnerabilities.

5. Secure Coding Practices

• Input Validation and Sanitization: validated and sanitized user inputs to prevent SQL injection and other code injection attacks.

• Use Secure Libraries and Frameworks: Chose well-maintained and secure libraries and frameworks for development.

• Avoiding Hard-Coding Secrets: Stored secrets like API keys and passwords in secure vaults or environment variables, not in the codebase.

• Server-side validation checks for valid characters and length restrictions (up to 40 alphanumeric characters) in search bar and prevents injection attacks and maintain data integrity.

6. User Privacy and Data Protection

• Data Minimization: Collected only the necessary user data and limit the storage period.

7. Secure User Interface

• Input Filtering: Implemented client-side input filtering to complement server-side validation.

• Escape Outputs: Ensure all user-generated content is properly escaped to prevent XSS.

8. Session Management

• Token-Based Authentication: Use tokens (e.g., JWT) for managing sessions securely.

9. Monitoring and Logging

• Regular Audits: Conducted regular security audits and vulnerability assessments to identify and address weaknesses.

10. Disaster Recovery and Incident Response

• Backup and Restore Procedures: Implemented regular backups and have a robust data recovery plan.

• Incident Response Plan: Developed and maintained a plan to quickly respond to security incidents.

By adhering to these security practices, ClassCompanion ensures robust protection of user credentials, data integrity, and application endpoints against common security threats. Regular updates and security audits further strengthen these measures to maintain a secure environment for users and data.

6. Self-check on Adherence to Original Non-functional Specifications:

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in Milestone 0. Application delivery shall be from chosen cloud server. **DONE**

2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers. **DONE**

3. All or selected application functions must render well on mobile devices. **DONE**

4. Data shall be stored in the database on the team's deployment cloud server. **DONE**

5. No more than 50 concurrent users shall be accessing the application at any time. **DONE**

6. Privacy of users shall be protected and all privacy policies will be appropriately communicated to the users. **DONE**

7. The language used shall be English (no localization needed). **DONE**

8. Application shall be very easy to use and intuitive. **DONE**

9. Application should follow established architecture patterns. **DONE**

10. Application code and its repository shall be easy to inspect and maintain. **DONE**

11. Google analytics shall be used (optional for Fulda teams). **NOT IMPLEMENTED**

12. No e-mail clients shall be allowed. **DONE**

13. Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI. **DONE**

14. Site security: basic best practices shall be applied (as covered in the class) for main data items. **DONE**

15. Application shall be media rich (images, video etc.). Media formats shall be standard as used in the market today. **DONE**

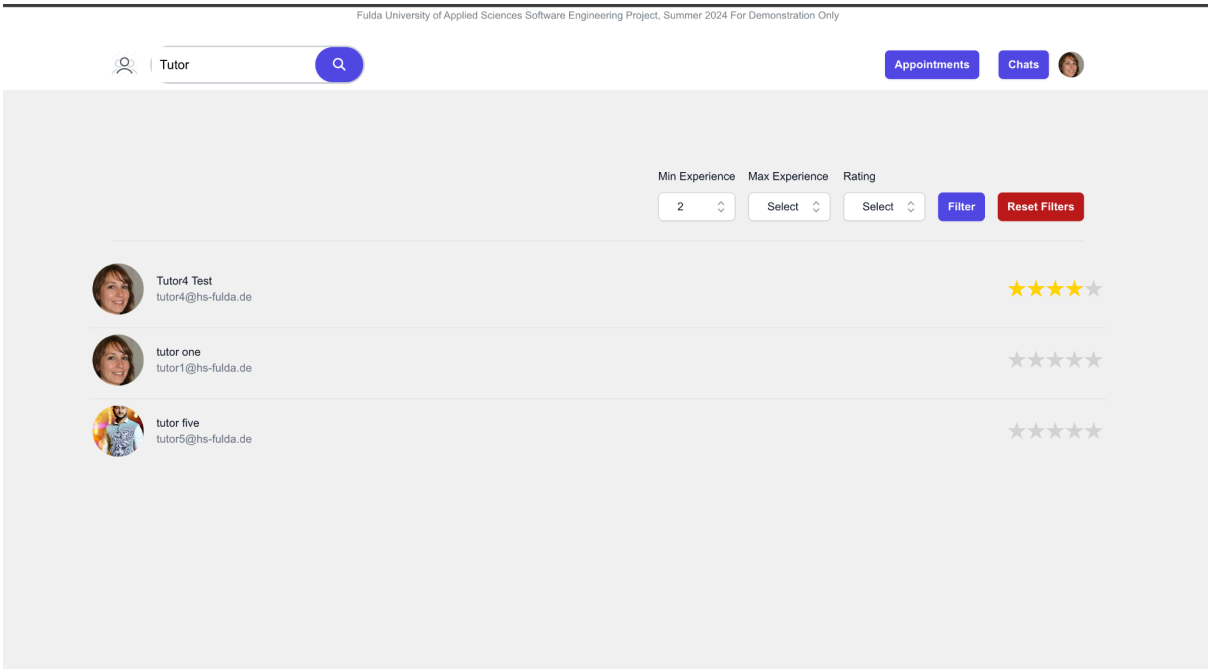
16. Modern SE processes and practices shall be used as specified in the class, including collaborative and continuous SW development. **DONE**

17. For code development and management, as well as documentation like formal milestones required in the class, each team shall use their own GitHub to be set-up by class instructors and started by each team during Milestone 0. **DONE**

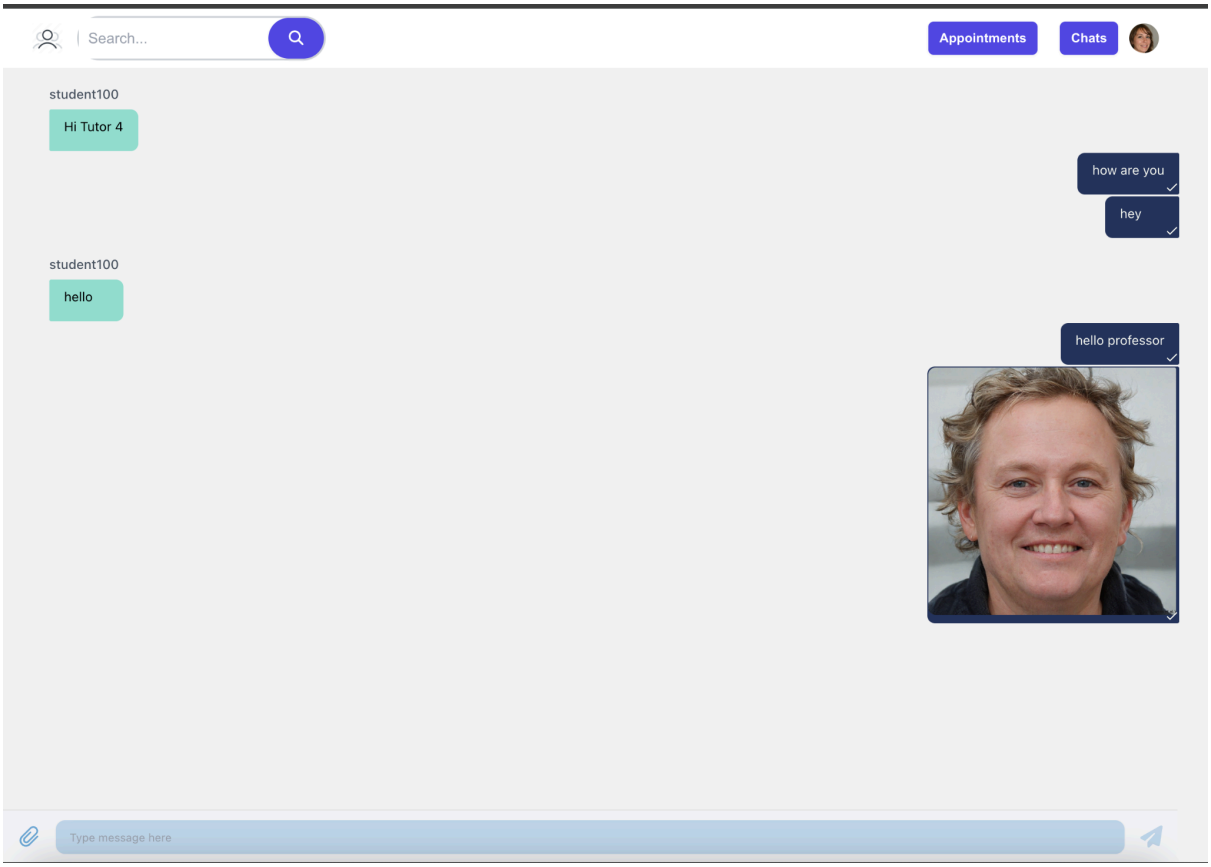
18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "Fulda University of Applied Sciences Software Engineering Project, Fall 2021 For Demonstration Only" at the top of the WWW page. (Important so as to not confuse this with a real application). **DONE**

ClassCompanion Screenshots

Tutor Search Page:



Chat Page:



Moderator Page:

